HTBL Krems

Höhere Lehranstalt für Informationstechnologie
Ausbildungsschwerpunkt: Systemtechnik

DIPLOMARBEIT

Anwendung für eine Firma

Ausgeführt im Schuljahr 2025/26 von:

Jacob Toifl	5AHIT-01
Michael Schaider	5AHIT-02

Betreuer:

Winkler Norbert, MSc
Winkler Norbert, MSc

Krems, am 01.04.2026

EIDESSTATTLICHE ERKLÄRUNG

Ich erkläre an Eides statt, dass ich die vorliegende Diplomarbeit selbstständig und ohne fremde Hilfe verfasst, andere als die angegebenen Quellen und Hilfsmittel nicht benutzt und die den benutzten Quellen wörtlich und inhaltlich entnommenen Stellen als solche erkenntlich gemacht habe.

Krems, (Datum)

Verfasser/Verfasserinnen:

Jacob Toifl

Michael Schaider

DIPLOMARBEIT

Bestätigung der Abgabe

Abgabebestätigung

Datum	Name	Unterschrift

Genehmigung der Diplomarbeit

Approbation

Datum	Prüfer*in	Abteilungsleiter*in Direktor*in

DIPLOMARBEIT

Dokumentation

Verfasser*innen
Jacob Toifl, 5AHIT
Michael Schaider, 5AHIT
Abteilung
Informationstechnologie
Ausbildungsschwerpunkt: Systemtechnik
Schuljahr
2025/2026
Thema der Diplomarbeit
Anwendung für eine Firma
Kooperationspartner
MBIT Solutions GmbH

Aufgabenstellung

Realisierung

Ergebnisse

DIPLOMA THESIS

Documentation

Authors
Jacob Toifl, 5AHIT
Michael Schaider, 5AHIT
Department
Information Technology
Specialization: Systems Engineering
Academic year
2025/2026
Thesis Topic
Application for a Company
Cooperation Partner
MBIT Solutions GmbH

Task Description

Implementation

Results

Inhaltsverzeichnis

1. Präambel 8
1.1. Kurzfassung . . . 8
1.2. Abstract . . . 8
1.3. Team . . . 8
1.4. Danksagung . . . 8
1.5. Gendererklärung . . . 9

2. Einleitung 10
2.1. Ausgangslage . . . 10
2.1.1. Systemarchitekturen und Berechtigungssysteme . . . 10
2.1.2. KI-gestützte Klassifizierung und Suchlogiken . . . 10
2.2. Forschungsfrage . . . 10
2.2.1. Dynamische Zugriffskontrolle und Skalierbarkeit von Dokumentensystemen . . . 10
2.2.2. Automatisierte Dokumentenanalyse und Suchoptimierung . . . 10

3. Theoretische Grundlagen 11
3.1. Künstliche Intelligenz zur Dokumentenverarbeitung . . . 11
3.1.1. KI-gestützte Dokumentenklassifikation . . . 11
3.1.2. Architekturen zur Integration externer KI-Dienstleister . . . 13
3.1.3. Merkmalsextraktion und Embedding . . . 15
3.1.4. Modellfamilien zur Dokumenttypenerkennung . . . 20
3.1.5. Datenaufbereitung und Preprocessing für Dokumente . . . 21
3.2. Architekturen für Dokumentensysteme . . . 24
3.2.1. Suche und Filter-Architekturen . . . 24
3.2.2. Rollen- und Berechtigungssysteme . . . 26
3.2.3. Skalierbare plattformunabhängige Systemarchitekturen . . . 29
3.2.4. Sharepoint . . . 31
3.2.5. Microsoft Entra ID . . . 33

4. Dokumentation der Implementierung 36
4.1. Dokumentation - Grundlegend . . . 36
4.1.1. Test Umgebung . . . 36
4.1.2. Technologien . . . 36
4.2. Dokumentation - Funktionen . . . 36
4.2.1. Dokumenten-Upload . . . 36
4.2.2. Dokumenten-Klassifikation . . . 42
4.2.3. Dokumenten-Suche . . . 45
4.2.4. Benutzer- und Rollenverwaltung . . . 49
4.2.5. System-Logging und Monitoring . . . 49
4.2.6. API-Endpunkte . . . 49
4.2.7. Fehlerbehandlung und Ausnahmen . . . 49
4.2.8. Sicherheitsfunktionen . . . 49

5. Beurteilung 50
5.1. Bewertung der Implementierung 50
5.2. Erfüllung der Forschungsfragen 50
5.3. Kritische Reflexion und Grenzen 50

6. Zusammenfassung und Ausblick 51
6.1. Zusammenfassung 51
6.2. Ausblick 51

I. Literaturverzeichnis 52

II. Abbildungsverzeichnis 56

III. Tabellenverzeichnis 57

IV. Quellcodeverzeichnis 58

A. Anhang 59
A.1. Arbeitsteilung 59
A.2. Kapitelverzeichnis 59
A.3. Projekttagebücher 59
A.3.1. Projekttagebuch Max Mustermann 59
A.3.2. Projekttagebuch Mex Musterjuan 59
A.4. Besprechungsprotokolle 60
A.5. Datenträgerbeschreibung 62

1. Präambel

1.1. Kurzfassung

Die vorliegende Diplomarbeit beschäftigt sich mit der Entwicklung und Implementierung des Systems DropIT, einer modernen Lösung zur strukturierten, sicheren und effizienten Verwaltung von Dokumenten innerhalb einer Organisation. Ziel des Projekts ist es, eine nutzerfreundliche Anwendung zu schaffen, welche den Upload, die Klassifizierung, die Suche sowie die Organisation von Dokumenten zentralisiert und vereinfacht. DropIT integriert sich nahtlos in bestehende Microsoft-Dienste wie SharePoint und Entra ID, wodurch sowohl private als auch unternehmensinterne Anwender von einer verbesserten Übersichtlichkeit, Automatisierung und Datensicherheit profitieren.

1.2. Abstract

DropIT is a user-friendly document management system developed to make it easier to store, organize, and find digital files. The system works together with Microsoft services such as SharePoint and Entra ID, allowing secure login and central storage of documents. With AI-supported classification and metadata extraction, DropIT can automatically recognize document types and important information like contract periods or expiration dates. Features such as full-text search, filters, and a built-in reminder system help users manage documents faster and more efficiently. This thesis describes the idea, design, development, and evaluation of the system. The goal of DropIT is to offer a clear, reliable, and scalable solution that improves document handling for both private users and organizations.

1.3. Team

Das Projektteam besteht aus:

- **Jacob Toifl** – Projektleiter
- **Michael Schaider** – Projektmitglied

1.4. Danksagung

Wir möchten uns an dieser Stelle herzlich bei allen Personen bedanken, die uns während der Erstellung dieser Diplomarbeit unterstützt haben. Besonderer Dank gilt unserem Betreuer **Winkler Norbert, MSc**, für seine fachliche Beratung, seine Unterstützung im Entwicklungsprozess und seine wertvollen Rückmeldungen. Ebenso bedanken wir uns bei der Firma **MBIT Solutions GmbH** für die tolle Zusammenarbeit und die Bereitstellung der notwendigen Ressourcen und Infrastruktur, die maßgeblich zum Erfolg dieses Projekts beigetragen haben.

1.5. Gendererklärung

Zur besseren Lesbarkeit der Diplomarbeit wurde ausschließlich die männliche Form verwendet. Da Begriffe wie „Benutzerinnen und Benutzer“ den Text unleserlich machen, wurde es schlicht auf „Benutzer“ gekürzt, dies soll jedoch keine Geschlechterdiskriminierung zum Ausdruck bringen.

2. Einleitung

2.1. Ausgangslage

2.1.1. Systemarchitekturen und Berechtigungssysteme

Das ist die Ausgangslage von Jacob Toifl.

2.1.2. KI-gestützte Klassifizierung und Suchlogiken

Das ist die Ausgangslage von Michael Schaider.

2.2. Forschungsfrage

2.2.1. Dynamische Zugriffskontrolle und Skalierbarkeit von Dokumentensystemen

Das ist die Forschungsfrage von Jacob Toifl.

2.2.2. Automatisierte Dokumentenanalyse und Suchoptimierung

Das ist die Forschungsfrage von Michael Schaider.

3. Theoretische Grundlagen

3.1. Künstliche Intelligenz zur Dokumentenverarbeitung

3.1.1. KI-gestützte Dokumentenklassifikation

3.1.1.1. Definition von Dokumentenklassifikation

Bei der Dokumentenklassifizierung werden Dokumente bestimmten, zuvor definierten Klassen zugeordnet. Das Dokument wird zunächst erfasst, anschließend werden die enthaltenen Informationen ausgelesen und ausgewertet. So lässt sich erkennen, um welche Art von Dokument es sich handelt, wo es abgelegt werden soll, welche Daten daraus übernommen werden müssen und in welchen Workflow es anschließend einfließen kann. [1]

Zum Einsatz kommen dabei unter anderem OCR und KI, die selbst sehr feine Unterschiede zwischen verschiedenen Dokumentarten identifizieren können. Mithilfe von OCR werden Textinhalte aus Bilddateien ausgelesen, automatisch kategorisiert und in eine strukturierte Form gebracht. Dadurch können Dokumente und ihre Inhalte effizient gespeichert, verwaltet, durchsucht und ausgewertet werden. [1]

Die Begriffe Dokumentklassifizierung und Textklassifizierung werden häufig synonym verwendet, weisen jedoch einige Unterschiede auf, wie in Tabelle 3.1 ersichtlich.

Aspekt	**Textklassifizierung**	**Dokumentenklassifizierung**
Geltungsbereich	Analysiert nur Textinhalt.	Analysiert Text sowie Layout- und Bildelemente.
Data Input	Rein textliche Daten (Sätze, Absätze).	Gesamtes Dokument inkl. Bilder und Tabellen.
Anwendungsfälle	Sentiment, Themenzuordnung, Spam-Erkennung.	Rechnungen, Verträge, Formulare.
Techniken	NLP-Methoden.	Kombination aus NLP, Computer Vision und OCR.

Tabelle 3.1.: Textklassifizierung vs. Dokumentenklassifizierung [2]

Im Allgemeinen lässt sich sagen, dass Textklassifizierung eine Teilmenge der Dokumentenklassifizierung ist, die sich ausschließlich auf den Textinhalt konzentriert, während die Dokumentenklassifizierung einen umfassenderen Ansatz verfolgt. [2]

3.1.1.2. Funktion der Dokumentenklassifizierung

Die Dokumentenklassifizierung kann grundsätzlich auf zwei Wegen erfolgen: manuell oder automatisiert. Bei der manuellen Klassifizierung prüft eine Person die Dokumente, identifiziert inhaltliche Zusammenhänge und ordnet sie anschließend den entsprechenden Kategorien zu. Bei der automatischen Dokumentenklassifizierung kommen hingegen Verfahren des maschinellen Lernens bzw. Deep Learnings zum Einsatz. Ziel ist es, Dokumente ohne menschliches Eingreifen systematisch zuzuordnen. Für betriebswirtschaftliche Anwendungen ist es daher wichtig, die unterschiedlichen Dokumentarten sowie die damit verbundenen Geschäftsprozesse zu verstehen. [2]

Strukturierte Dokumente weisen klar definierte, einheitlich formatierte Daten auf (z. B. konsistente Nummerierung, Schriftarten und Layouts). Aufgrund dieser hohen Standardisierung lassen sich Klassifizierungsmodelle für solche Unterlagen vergleichsweise einfach entwickeln und die Ergebnisse sind gut prognostizierbar. [2]

Unstrukturierte Dokumente liegen in einem freien, wenig standardisierten Format vor. Beispiele sind Schreiben, Verträge oder Bestellungen mit variierendem Aufbau und sprachlicher Gestaltung. Durch diese Heterogenität ist die automatisierte Identifikation relevanter Informationen deutlich komplexer, was den Einsatz leistungsfähiger Klassifikationsverfahren erforderlich macht. [2]

3.1.1.3. Funktion der KI-basierten Dokumentenklassifizierung

Die automatisierte Klassifizierung von Dokumenten mit KI erfolgt typischerweise in mehreren aufeinanderfolgenden Schritten:

1. **Datensammlung und Beschriftung**
   Die Basis sind hochwertige, breit gefächerte Datenbestände. Dazu werden Dokumente aus unterschiedlichen Kategorien gesammelt und sauber mit passenden Labels versehen, damit Machine-Learning-Modelle sinnvoll trainiert werden können. [2]
2. **Vorverarbeitung und Feature-Erzeugung**
   Liegt ein Dokument als Scan oder Bild vor, wird der enthaltene Text zunächst per OCR (optische Zeichenerkennung) ausgelesen. Anschließend bereinigen NLP-Verfahren den Text, zerlegen ihn in Tokens und überführen ihn in aussagekräftige Merkmalsrepräsentationen. Parallel dazu wertet Computer Vision das Seitenlayout und visuelle Strukturen aus. [2]
3. **Training des Klassifikationsmodells**
   Überwachte Lernverfahren (etwa Transformer-Modelle oder CNNs) werden mit den gelabelten Beispielen trainiert, um wiederkehrende Muster zu entdecken. Das Modell lernt dabei, die gewonnenen Merkmale den jeweiligen Dokumentkategorien zuzuordnen. [2]

4. **Evaluation und Feintuning**
   Im Anschluss wird das Modell mit bislang unbekannten Testdaten geprüft, um Kennzahlen wie Genauigkeit, Präzision und Recall zu bestimmen. Durch Anpassung von Hyperparametern und ggf. Modellvarianten wird die Performance weiter verbessert. [2]
5. **Produktivbetrieb und laufende Anpassung**
   Nach der Implementierung ordnet das Modell neue Dokumente automatisch in Echtzeit den passenden Klassen zu. Über Nutzerfeedback und zusätzliche Trainingsdaten wird es regelmäßig nachtrainiert und kann seine Treffgenauigkeit im Zeitverlauf kontinuierlich steigern. [2]

3.1.2. Architekturen zur Integration externer KI-Dienstleister

3.1.2.1. Grundidee der Integration externer KI-Dienste

Externe KI-Dienstleister stellen ihre Modelle typischerweise als verwaltete Cloud- oder API-Dienste bereit. Für Unternehmen reduziert das den Aufwand für den Eigenbetrieb komplexer Modelle, beschleunigt die Einführung neuer KI-Funktionen und ermöglicht einen schnellen Zugriff auf aktuelle Modellgenerationen. Gleichzeitig verlagert sich ein Teil der Systemarchitektur von der Modellimplementierung hin zur sauberen Integration externer Abhängigkeiten, etwa für Authentifizierung, Quotensteuerung, Fehlertoleranz und Kostenkontrolle. [3, 4]

3.1.2.2. Direkte Kopplung und Vermittlungsschicht

Die einfachste Integrationsform ist die direkte Kopplung einer Anwendung an den Endpunkt eines externen KI-Anbieters. Dieses Muster eignet sich vor allem für Prototypen oder kleine Lösungen mit nur wenigen Konsumenten. Mit zunehmender Systemgröße entstehen jedoch Nachteile: Modellwechsel, neue Endpunkte oder geänderte Authentifizierungsverfahren müssen direkt in den konsumierenden Anwendungen umgesetzt werden. Eine vorgeschaltete Vermittlungsschicht in Form eines Gateways oder eines internen KI-Adapters entkoppelt daher die Clients vom konkreten Anbieter und erlaubt serverseitige Routing-Entscheidungen, ohne Client-Code oder Konfiguration neu ausrollen zu müssen. [5]

Ein zentrales Gateway übernimmt in diesem Zusammenhang nicht nur die Weiterleitung von Requests, sondern auch Governance-Aufgaben. Laut Microsoft Learn kann ein AI-Gateway Authentifizierung und Autorisierung bündeln, Anfragen über mehrere KI-Endpunkte verteilen, Interaktionen überwachen, Tokenverbrauch begrenzen und verschiedene Anbieter über eine gemeinsame Verwaltungsschicht anbinden. Dadurch entsteht eine providernahe, aber fachlich zentralisierte Integrationsarchitektur, die besonders in größeren Organisationen die Standardisierung vereinfacht. [4]

3.1.2.3. Asynchrone Integrationsmuster

Für dokumentenbezogene KI-Anwendungsfälle ist eine rein synchrone Kommunikation oft nicht ausreichend. OCR, Klassifikation, Extraktion oder mehrstufige Prompt-Ketten können

zu längeren Laufzeiten führen, die eine direkte Request-Response-Kommunikation unnötig blockieren würden. Das Architekturpattern „Asynchronous Request-Reply“ entkoppelt deshalb die Annahme einer Anfrage von ihrer eigentlichen Verarbeitung: Der Client erhält zunächst eine Bestätigung, während die Verarbeitung in einer Queue oder einer nachgelagerten Komponente stattfindet und der Status später über Polling, Redirects oder Benachrichtigungsmechanismen abgefragt wird. Dadurch steigen Reaktionsfähigkeit, Parallelität und Skalierbarkeit der Gesamtlösung. [6]

3.1.2.4. Sicherheit, Governance und Datenhoheit

Sobald externe KI-Dienstleister in produktive Systeme eingebunden werden, wird die Sicherheitsarchitektur zu einem eigenständigen Entwurfsbereich. Ein Gateway erlaubt es, Zugangsdaten an zentraler Stelle zu terminieren und gegenüber dem externen Dienst mit einer eigenen Identität aufzutreten. Microsoft empfiehlt hierfür insbesondere Managed Identities, damit API-Schlüssel nicht in Anwendungen, Konfigurationsdateien oder Quellcode verteilt werden müssen. Darüber hinaus lassen sich über die Vermittlungsschicht Protokollierung, Content-Safety-Policies und feinere Zugriffskontrollen umsetzen. [4, 5]

Hinzu kommt die Frage der Datenhoheit. Bei KI-Services, die über mehrere Instanzen, Regionen oder Anbieter verteilt werden, müssen Architekturentscheidungen auch unter Compliance- und Residency-Gesichtspunkten betrachtet werden. Microsoft weist ausdrücklich darauf hin, dass active-active- oder active-passive-Strategien nur innerhalb der zulässigen geopolitischen Grenzen angewendet werden sollten, wenn Datenresidenz eine Anforderung darstellt. Für sensible Dokumentensysteme bedeutet das, dass Routing- und Failover-Logik nicht nur nach Verfügbarkeit, sondern auch nach regulatorischen Vorgaben entworfen werden muss. [5]

3.1.2.5. Resilienz gegenüber Anbieterfehlern

Externe KI-Dienste sind aus Sicht der eigenen Anwendung entfernte Ressourcen und damit grundsätzlich anfällig für temporäre Fehler wie Timeouts, Überlastung oder kurzzeitige Nichterreichbarkeit. Das Retry-Pattern adressiert solche vorübergehenden Störungen, indem fehlgeschlagene Aufrufe kontrolliert wiederholt werden. Die Dokumentation des Azure Architecture Center empfiehlt dabei insbesondere verzögerte und bei Bedarf exponentiell ansteigende Wiederholungen, um ausgelastete Dienste nicht zusätzlich zu belasten. Für Integrationen mit KI-Dienstleistern ist diese Logik zentral, weil Lastspitzen und Quotenengpässe in der Praxis regelmäßig auftreten können. [7]

Nicht jeder Fehler ist jedoch nur kurzfristig. Dauert eine Störung länger an, reicht Wiederholen allein nicht aus. Das Circuit-Breaker-Pattern verhindert in diesem Fall weitere Aufrufe an eine wahrscheinlich fehlerhafte Abhängigkeit und ermöglicht so eine sanfte Degradation des Gesamtsystems. Gerade bei der Einbindung externer KI-APIs schützt dieses Muster davor, dass ein gestörter Anbieter die gesamte Anwendung blockiert oder nachgelagerte Komponenten überlastet. Microsoft beschreibt explizit die Kombination von Retry und Circuit Breaker als robusten Ansatz zur Fehlerbehandlung in verteilten Systemen. [8, 7]

3.1.2.6. Performance-, Kosten- und Routingoptimierung

Neben Sicherheit und Resilienz spielt bei externen KI-Dienstleistern die effiziente Nutzung von Tokens, Durchsatz und Latenz eine zentrale Rolle. Ein Gateway kann hierfür Tokenlimits pro Konsument durchsetzen, Last über mehrere Backends verteilen und semantisches Caching einsetzen, um wiederkehrende Anfragen nicht erneut vollständig an den Provider zu senden. Auf Anbieter-Seite beschreibt OpenAI ergänzend, dass sich Latenzen architektonisch unter anderem durch weniger Requests, Parallelisierung unabhängiger Schritte, kleinere Modelle für Teilaufgaben und klassische Optimierungstechniken reduzieren lassen. Damit wird deutlich, dass die Integrationsarchitektur nicht nur eine Verbindungsschicht ist, sondern aktiv die Wirtschaftlichkeit und Antwortzeit von KI-Funktionen beeinflusst. [4, 9]

Ein weiterer Optimierungshebel ist Caching auf Prompt-Ebene. OpenAI dokumentiert, dass Prompt Caching bei langen, wiederkehrenden Präfixen Latenz und Eingabekosten deutlich senken kann, wenn statische Instruktionen am Anfang und variable Inhalte am Ende eines Prompts angeordnet werden. In Verbindung mit einem zentralen Gateway oder einem internen KI-Service entsteht daraus ein zweistufiges Optimierungsmodell: Einerseits kann die Integrationsschicht Anfragen deduplizieren oder semantisch cachen, andererseits nutzt der externe Anbieter dieselben oder ähnliche Prompt-Präfixe effizienter. [10, 4]

3.1.2.7. Einordnung für Dokumentenverarbeitungssysteme

Für Systeme zur Dokumentenverarbeitung ergibt sich daraus ein klares Zielbild: Externe KI-Dienstleister sollten nicht direkt und unkontrolliert aus jeder Fachkomponente heraus aufgerufen werden, sondern über eine definierte Integrationsarchitektur mit zentralem Routing, Sicherheitskontrollen, Quotenmanagement und Beobachtbarkeit. Ergänzt um asynchrone Verarbeitungsmuster, Retry-/Circuit-Breaker-Mechanismen sowie providernahe Caching- und Routingstrategien entsteht eine Architektur, die sowohl technisch robust als auch organisatorisch beherrschbar bleibt. Genau diese Entkopplung ist entscheidend, wenn KI-Funktionen langfristig in produktive Dokumenten- und Unternehmensprozesse eingebettet werden sollen. [3, 4, 6, 8]

3.1.3. Merkmalsextraktion und Embedding

3.1.3.1. Definition von Merkmalsextraktion

Die Merkmalsextraktion ist ein wichtiger Schritt in der Datenanalyse und im maschinellen Lernen. Dabei werden aus Rohdaten gezielt die Informationen herausgefiltert, die für eine spätere Auswertung oder ein Modell relevant sind. Ziel ist es, aussagekräftige Merkmale hervorzuheben und unwichtige oder störende Anteile zu reduzieren, sodass die Daten kompakter und besser nutzbar werden. [11]

Je nach Datentyp kommen dafür unterschiedliche Methoden zum Einsatz – von einfachen statistischen Verfahren bis hin zu maschinellen Lernverfahren, die Muster automatisch erkennen. Die gewonnenen Merkmale werden häufig in einer strukturierten Form zusammengefasst (z.

B. als Merkmalsvektor) und bilden dann die Grundlage für weitere Schritte wie Klassifikation oder Vorhersagen. [11]

Wie in Tabelle 3.2 ersichtlich, werden im Folgenden wichtige Begriffe der Merkmalsextraktion mit kurzer Beschreibung dargestellt.

Begriff	**Beschreibung**
Merkmal	Eine quantitativ oder qualitativ erfassbare Eigenschaft, die ein Datenobjekt beschreibt.
Feature Extraction	Verfahren, bei dem aus Rohdaten gezielt die aussagekräftigen Eigenschaften herausgelöst bzw. abgeleitet werden.
Vektor	Geordnete Sammlung von Merkmalwerten, die ein Objekt in strukturierter Form repräsentiert.
Dimensionalität	Anzahl der enthaltenen Merkmale bzw. Einträge eines Vektors.

Tabelle 3.2.: Wichtige Begriffe der Merkmalsextraktion [11]

3.1.3.2. Funktion der KI-gestützten Merkmalsextraktion

KI-gestützte Merkmalsextraktion ist ein zentraler Bestandteil moderner Datenanalyse. Dabei werden automatisierte Verfahren eingesetzt, um aus großen Datensätzen gezielt relevante Informationen herauszuarbeiten. Durch den Einsatz unterschiedlicher KI-Algorithmen können Muster und Zusammenhänge präzise erkannt und die Daten effizient für weitere Analysen oder Modelle aufbereitet werden. [11]

Die in Tabelle 3.3 aufgeführten KI-Algorithmen sind zentrale Methoden für die Merkmalsextraktion.

Algorithmus	**Beschreibung**
Support Vector Machines (SVM)	Geeignet für Klassifikation und Regression; bestimmt eine optimale Trennlinie bzw. Entscheidungsgrenze zwischen Datenpunkten.
Decision Trees	Erstellen Entscheidungsbäume anhand von Merkmalen und Regeln, um Vorhersagen oder Klassen zu bestimmen.
Random Forest	Kombiniert viele Entscheidungsbäume und erhöht dadurch meist Genauigkeit und Stabilität der Ergebnisse.

Tabelle 3.3.: Relevante Algorithmen zur Merkmalsextraktion [11]

Maschinelles Lernen unterstützt die Merkmalsextraktion, indem Modelle aus Trainingsdaten lernen, welche Merkmale für eine Aufgabe wichtig sind, und weniger relevante Informationen ausblenden. Für Textdaten ist besonders Natural Language Processing (NLP) relevant, da damit Bedeutungen und Zusammenhänge in Sprache erkannt werden können.

Wichtige Methoden sind Tokenisierung, Lemmatisierung und Named Entity Recognition (NER). Moderne Modelle wie BERT und GPT-3 nutzen große Datenmengen, um Muster im Kontext zu erfassen und Texte präziser zu analysieren. [11]

Text Mining ist ein Teilbereich der Merkmalsextraktion, der sich auf unstrukturierte Textdaten konzentriert. Ziel ist es, Texte computergestützt so auszuwerten, dass relevante Informationen herausgefiltert und unwichtige Inhalte reduziert werden, um daraus verwertbare Erkenntnisse zu gewinnen. Dabei werden insbesondere Verfahren des maschinellen Lernens und der Natural Language Processing (NLP) eingesetzt, um Muster, Bedeutungen und Zusammenhänge in großen Textmengen zu erkennen. Typische Anwendungsfelder sind unter anderem die Analyse von Kundenkommunikation sowie der Einsatz in Forschungskontexten, z. B. zur automatisierten Durchsicht umfangreicher Dokumente. [11]

3.1.3.3. Definition von Embedding

Ein Embedding ist eine Darstellung von Objekten wie Text, Bildern oder Audio als Zahlenvektoren in einem kontinuierlichen Vektorraum. Dabei sind ähnliche Inhalte im Raum näher beieinander angeordnet, sodass Machine-Learning-Modelle Ähnlichkeiten und Zusammenhänge leichter erkennen können. [12]

Embeddings werden in vielen Anwendungen genutzt, z. B. für Suche, Empfehlungssysteme, Chatbots oder Betrugserkennung. Im Unterschied zu handgebauten Merkmalen entstehen sie meist automatisch durch Lernverfahren (z. B. neuronale Netze), die Muster und Beziehungen direkt aus den Daten ableiten. Dadurch können Modelle nicht nur einzelne Wörter oder Elemente isoliert betrachten, sondern auch deren Kontext und Bedeutung besser erfassen. [12]

3.1.3.4. Prinzip und Funktionsweise von Embeddings

Bei der Embedding-Funktionsweise werden Rohdaten zunächst in ein numerisches Format überführt, weil viele ML-Algorithmen nur mit Zahlen arbeiten (z. B. Text als Bag-of-Words, Bilder als Pixelwerte oder Graphdaten als Matrix). Ein Embedding-Modell erzeugt daraus Vektoren – also Zahlenlisten –, die ein Objekt als Punkt in einem hochdimensionalen Raum repräsentieren. Jede Zahl steht für die Position entlang einer Dimension; je nach Aufgabe können es sehr viele Dimensionen sein. Ähnliche Objekte liegen im Vektorraum näher beieinander, und diese Nähe wird mit Ähnlichkeitsmaßen wie Cosinus-Ähnlichkeit oder euklidischer Distanz bewertet. [12]

In Abbildung 3.1 ist dieses Prinzip schematisch dargestellt: semantisch ähnliche Begriffe liegen im Vektorraum näher beieinander (z. B. Cat, Dog, Wolf), während thematisch andere Begriffe weiter entfernt positioniert sind (z. B. Apple, Banana). Die räumliche Distanz dient damit als Maß für Ähnlichkeit.

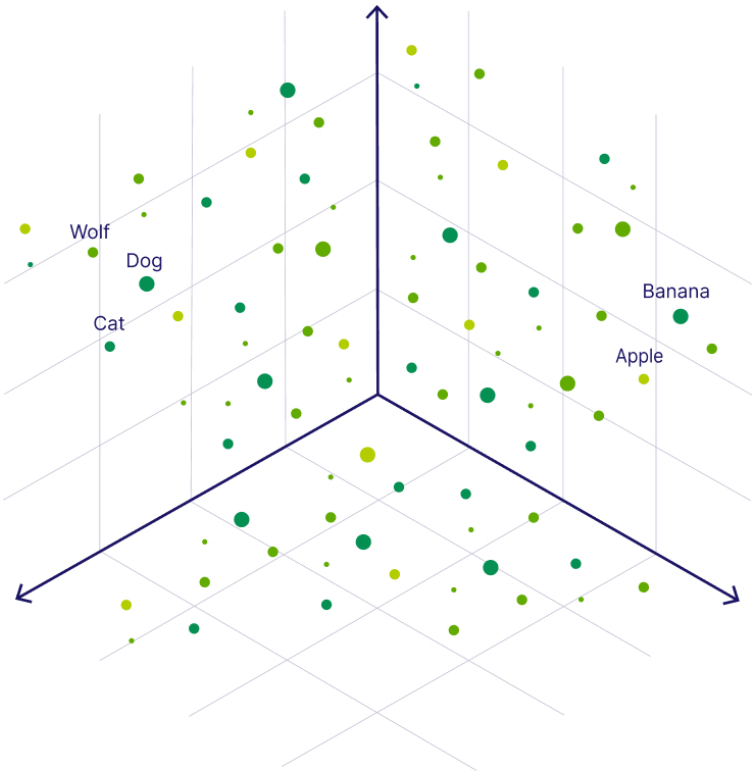

Abbildung 3.1.: Übersicht der Vektordarstellung von Embeddings [13]

Ein anschauliches Beispiel sind Wort-Embeddings: Wörter werden als Vektoren dargestellt, etwa „Papa“ und „Mama“. Auch wenn beide inhaltlich verwandt sind, wäre zu erwarten, dass „Vater“ im Vektorraum noch näher an „Papa“ liegt als „Mama“, weil die Bedeutung stärker übereinstimmt. [12]

Beispielhaft als Vektoren:

- „Papa“ = [0, 1548, 0, 4848, . . . , 1, 864]
- „Mama“ = [0, 8785, 0, 8974, . . . , 2, 794]

In Empfehlungssystemen werden sowohl Nutzer als auch Artikel als Embedding-Vektoren gelernt. Die Grundidee: Ein Nutzer und ein Artikel passen umso besser zusammen, je größer das Punktprodukt ihrer Vektoren ist. Der Empfehlungsscore wird dabei typischerweise so berechnet:

$$score = u * i$$

mit

- Empfehlungsscore $score$,
- Embedding des Nutzers u,
- Embedding des Artikels i,

Im Training werden diese Embeddings anhand historischer Interaktionen (z. B. Klicks, Käufe, Bewertungen) angepasst, sodass hohe Scores mit tatsächlichen Präferenzen möglichst gut übereinstimmen. Danach können Artikel mit den höchsten Scores als Top-N-Empfehlungen ausgegeben werden. [12]

3.1.3.5. Häufige Objekte für Embeddings

Embeddings sind flexible Repräsentationen, die sich auf unterschiedliche Datentypen anwenden lassen. Die häufigsten Objekte, die eingebettet werden können, sind in Tabelle 3.4 dargestellt.

Objekt	**Beschreibung**
Wörter	Dichte Vektorrepräsentationen einzelner Wörter, die Bedeutung und Kontextbeziehungen im Sprachkorpus abbilden.
Text	Vektoren für ganze Texte, die den Gesamtinhalt semantisch zusammenfassen und Vergleiche/Klassifikation erleichtern.
Bilder	Embeddings, die visuelle Merkmale und Bildinhalt als Vektor kodieren, z. B. für Ähnlichkeitssuche oder Objekterkennung.
Audio	Vektorrepräsentationen relevanter Klang- und Sprachmerkmale, nutzbar für Spracherkennung, Klassifikation oder Musikanalyse.
Graphen	Embeddings für Knoten oder ganze Netzwerke, die Struktur und Beziehungen erfassen, z. B. für Link Prediction oder Community Detection.

Tabelle 3.4.: Relevante Objekte für Embeddings [12]

3.1.3.6. Abgrenzung: Merkmalsextraktion vs. Embedding

Merkmalsextraktion beschreibt den allgemeinen Prozess, aus Rohdaten aussagekräftige Merkmale zu gewinnen oder abzuleiten, sodass die Daten für Analyse- oder Lernverfahren besser nutzbar sind. Dabei können unterschiedliche Methoden zum Einsatz kommen (z. B. statistische Kennwerte, regelbasierte Verfahren oder lernbasierte Ansätze), und das Ergebnis sind Merkmale, die eine Aufgabe gezielt unterstützen (z. B. Klassifikation oder Vorhersage).

Ein Embedding ist dagegen eine spezielle Form der Merkmalsrepräsentation: Objekte wie Wörter, Sätze, Bilder oder Nutzer/Artikel werden als dichte Vektoren in einem kontinuierlichen Vektorraum dargestellt. Die zentrale Idee ist, dass semantisch oder inhaltlich ähnliche Objekte im Vektorraum näher beieinanderliegen und dadurch über Ähnlichkeitsmaße vergleichbar werden.

3.1.4. Modellfamilien zur Dokumenttypenerkennung

Die Dokumenttypenerkennung ist ein zentraler Baustein in der KI-gestützten Dokumentenverarbeitung. Ziel ist es, eingehende Dokumente – etwa Rechnungen, Angebote, Verträge oder Formulare – automatisch einem definierten Dokumenttyp zuzuordnen, um nachgelagerte Verarbeitungsschritte wie Routing, Extraktion oder Validierung zu steuern. Je nach Anwendungsfall stehen dafür unterschiedliche Modellansätze zur Verfügung, die sich vor allem darin unterscheiden, welche Informationsquellen eines Dokuments sie nutzen: reinen Textinhalt, visuelle Merkmale einer Seite, Layout- und Positionsinformationen oder Kombinationen daraus. In der Literatur und in praktischen Systemen haben sich daher mehrere „Modellfamilien“ etabliert.

3.1.4.1. Textbasierte Modellfamilie

Textbasierte Verfahren formulieren die Dokumenttypenerkennung als Textklassifikationsproblem: Ein Dokument wird zunächst in eine rein textuelle Darstellung überführt (z. B. durch direkt extrahierbaren PDF-Text oder durch OCR bei gescannten Dokumenten). Anschließend wird dieser Text einer vordefinierten Klasse zugeordnet. Methodisch entspricht dies dem allgemeinen Vorgehen der Textklassifikation, bei dem Textinhalte analysiert und mittels gelabelter Beispiele in Kategorien eingeteilt werden. [14]

Innerhalb dieser Modellfamilie werden sowohl klassische Feature-basierte Ansätze (z. B. Bag-of-Words/gewichtete Termrepräsentationen) als auch neuere neuronale Modelle eingesetzt. Moderne Systeme nutzen häufig kontextuelle Sprachmodelle (Transformer), um neben Schlüsselbegriffen auch semantische Zusammenhänge zu berücksichtigen. Die Leistungsfähigkeit textbasierter Ansätze hängt in der Dokumentenverarbeitung jedoch stark von der Qualität der Texterfassung ab: OCR-Fehler, fehlende Sonderzeichen oder fragmentierter Text können die Klassifikation erschweren. Außerdem werden Layout- und Strukturinformationen eines Dokuments (z. B. Positionen von Feldern oder tabellarische Anordnungen) in einem rein textuellen Ansatz nur indirekt oder gar nicht abgebildet. [14]

3.1.4.2. Bildbasierte Modellfamilie

Bildbasierte Verfahren behandeln Dokumenttypenerkennung als Bildklassifikation: Seiten werden als gerenderte PDF-Seiten oder Scans (Seitenbilder) verarbeitet und auf Basis visueller Merkmale klassifiziert. Solche Ansätze nutzen typischerweise Convolutional Neural Networks (CNNs), die darauf ausgelegt sind, Muster in visuellen Daten zu erkennen und sich besonders für Klassifikationsaufgaben in der Computer Vision eignen. [15]

Ein wichtiges Argument für bildbasierte Ansätze ist, dass Dokumenttypen oft durch wiederkehrende visuelle Strukturen unterscheidbar sind (z. B. Formularcharakter, Briefkopf, typische Anordnung von Blöcken). In der Forschung wird die Dokumentbildklassifikation häufig auf etablierten Datensätzen evaluiert; als verbreitetes Beispiel gilt RVL-CDIP mit 16 Dokumentklassen und insgesamt 400.000 Dokumentbildern. Neben CNNs kommen zunehmend Vision Transformer (ViT) zum Einsatz, die Bilder in Patches zerlegen und diese ähnlich wie Token-Sequenzen verarbeiten, um globale Zusammenhänge besser abzubilden. [16] [17]

3.1.4.3. Layout-aware / dokumentenspezifische Transformer

Layout-aware Modelle erweitern textbasierte Ansätze um räumliche Layoutinformationen. Statt Text nur als Sequenz zu betrachten, wird zusätzlich modelliert, wo ein Token auf der Seite steht (z. B. über Bounding Boxes). Damit lässt sich die Dokumenttypenerkennung als Aufgabe formulieren, bei der Inhalt und Struktur gemeinsam berücksichtigt werden (z. B. typische Positionen von Überschriften, Adressblöcken, Tabellenbereichen). [18]

Ein prominentes Beispiel für diese Modellfamilie ist die LayoutLM-Klasse: In der Implementierung (z. B. in verbreiteten Frameworks) wird explizit ein zusätzliches Eingabesignal "bbox" erwartet, das die Bounding Boxes der Tokens enthält und damit die 2D-Positionen abbildet. [18]

3.1.4.4. Multimodale Foundation-Model-Familie (VLM/LLM)

Multimodale Foundation-Model-Ansätze nutzen große Modelle, die visuelle und häufig auch sprachliche Informationen gemeinsam verarbeiten können. In der Dokumenttypenerkennung werden Dokumentseiten (als Bild bzw. gerendertes PDF) typischerweise zusammen mit einer natürlichsprachlichen Aufgabenbeschreibung (Prompt) an ein Vision-Language-Modell übergeben, das den Dokumenttyp direkt ausgibt. Solche Verfahren sind besonders dann attraktiv, wenn eine schnelle Erweiterung auf neue Klassen erforderlich ist oder wenn man Zero-/Few-shot-Strategien erproben möchte. [19]

Dass diese Vorgehensweise praktisch umgesetzt wird, zeigen Anbieterbeispiele: Ein öffentliches Azure-Sample demonstriert explizit die Klassifikation eines Dokuments mithilfe eines Vision-fähigen GPT-4o-Modells (Prompt + Dokumentbild), also genau das Grundprinzip dieser Modellfamilie. [19]

3.1.5. Datenaufbereitung und Preprocessing für Dokumente

Die Datenaufbereitung ist ein zentraler Erfolgsfaktor in der KI-gestützten Dokumentenverarbeitung, weil Klassifikationsmodelle nur so gut sind wie die Qualität ihrer Eingabedaten. Bereits kleine Inkonsistenzen in Scanqualität, Dokumentstruktur oder Labeling wirken sich direkt auf Modellgüte, Generalisierbarkeit und spätere Produktionsstabilität aus. [2]

Gerade bei heterogenen Dokumentbeständen (z. B. gescannte Belege, digital erzeugte PDFs und Formularbilder) ist Preprocessing daher kein optionaler Zwischenschritt, sondern eine eigene Prozessphase. Sie verbindet die Rohdatenerfassung mit der modellgerechten Repräsentation und reduziert Informationsverluste über die gesamte Pipeline hinweg. [2]

3.1.5.1. Zielsetzung und Qualitätskriterien

Das Ziel der Dokumentaufbereitung besteht darin, Eingaben technisch zu standardisieren, semantisch verwertbar zu machen und gleichzeitig den Informationsgehalt für nachgelagerte Modelle zu erhalten. In der Praxis bedeutet das vor allem: robuste OCR-Ergebnisse, stabile Layoutsignale und konsistente Trainingslabels. [2]

Aus OCR-Sicht hängt die Lesegenauigkeit stark von Eigenschaften wie Auflösung, Schräglage, Rauschen, Binarisierung und Randartefakten ab. Die Tesseract-Dokumentation beschreibt dafür konkrete Vorverarbeitungsschritte wie Rescaling (typisch mindestens 300 dpi), Binarisierung, Rauschreduktion und Deskewing, um die Erkennungsqualität zu erhöhen. [20]

Für dokumentenspezifische Transformer reicht reine Textqualität jedoch nicht aus, weil zusätzlich Layoutinformationen benötigt werden. Modelle wie LayoutLM verarbeiten deshalb nicht nur Tokens, sondern auch räumliche Positionsinformationen (Bounding Boxes), damit Inhalte im Seitenkontext interpretiert werden können. [21] [18]

Die wichtigsten Qualitätsziele der Aufbereitung sind in Tabelle 3.5 zusammengefasst.

Qualitätsziel	**Typische Maßnahme**	**Nutzen für KI-Modelle**
Lesbarkeit des Dokuments	Rescaling, Binarisierung, Rauschfilterung	Stabilere OCR-Texte und weniger fehlerhafte Tokens
Räumliche Konsistenz	Deskewing, Randkorrektur, Box-Normalisierung	Bessere Nutzung von Layoutmerkmalen für Dokumentverständnis
Semantische Konsistenz	Einheitliches Labeling und Klassenregeln	Weniger Labelrauschen, höhere Klassifikationsgüte

Tabelle 3.5.: Qualitätsziele in der Dokumentaufbereitung [20] [21]

3.1.5.2. Technische Preprocessing-Pipeline

Eine belastbare Pipeline folgt typischerweise einem klaren Ablauf von Standardisierung, visueller Korrektur, Textextraktion, Feature-Aufbereitung und Datensplitting. Damit wird sichergestellt, dass sowohl klassische textbasierte Modelle als auch multimodale Architekturen mit konsistenten Eingaben trainiert werden. [2] [22]

1. **Format- und Eingangsstandardisierung**
   Zu Beginn werden Dateiformate vereinheitlicht (z. B. Scanbilder und PDFs in eine verarbeitbare Seitenrepräsentation), Seitenmetriken geprüft und ungeeignete Dokumente ausgeschlossen. Für OCR-basierte Verarbeitung ist insbesondere eine ausreichende Bildauflösung relevant; Tesseract nennt hier mindestens 300 dpi als sinnvollen Richtwert. [20]
2. **Bildvorverarbeitung**
   Anschließend folgt die visuelle Bereinigung mit Binarisierung, Entfernung von Bildrauschen, Korrektur von Schräglagen (Deskewing) und Behandlung von Randartefakten. Diese Schritte verbessern die Zeichen- und Zeilensegmentierung und senken damit die Wahrscheinlichkeit systematischer OCR-Fehler. [20]
3. **OCR und Strukturerfassung**
   Nach der Bildbereinigung wird der Text extrahiert und gleichzeitig die Seitenstruktur erfasst (z. B. Positionen von Wörtern oder Blöcken). Für layoutbewusste Modelle ist diese Strukturinformation essenziell, da Textinhalte erst in Kombination mit Position und Kontext ihre volle Aussagekraft entfalten. [18] [21]

4. **Textnormalisierung und Feature-Aufbereitung**
   Auf OCR-Ebene werden üblicherweise Normalisierungsschritte wie Bereinigung von Artefaktzeichen, Vereinheitlichung von Datums-/Zahlformaten und konsistente Tokenrepräsentationen angewendet. Dadurch sinkt die Streuung technischer Schreibvarianten, und die Merkmale werden für das Klassifikationsmodell robuster. [2]
5. **Multimodale Repräsentation**
   In modernen Dokument-Pipelines werden Text-, Layout- und Bildsignale gemeinsam verarbeitet. LayoutLMv2 zeigt, dass die gemeinsame Modellierung dieser Modalitäten auf mehreren Dokumentaufgaben deutliche Leistungsgewinne gegenüber rein textuellen Ansätzen ermöglicht. [22]
6. **Training/Validierung/Test-Split**
   Vor dem Training werden die Daten reproduzierbar in Teilmengen aufgeteilt (z. B. mit fixer Zufallsinitialisierung und stratifizierter Verteilung), um Verzerrungen bei der Evaluation zu reduzieren. Für komplexe Datenstrukturen mit Gruppenzugehörigkeiten (z. B. mehrere Seiten desselben Dokuments) sind gruppenbasierte Splitter sinnvoll, damit keine Informationsleckage zwischen Train- und Testmenge entsteht. [23] [24]

3.1.5.3. Datenannotation, Balancierung und Leakage-Vermeidung

Neben der technischen Vorverarbeitung ist die semantische Qualität der Labels entscheidend. In der Dokumentklassifikation sollten Klassenregeln eindeutig dokumentiert, Label-Guidelines versioniert und Grenzfälle (z. B. Mischdokumente) konsistent behandelt werden, damit das Modell keine widersprüchlichen Lernsignale erhält. [2]

Ein weiteres Risiko ist Klassenungleichgewicht, das in realen Unternehmensdaten häufig vorkommt (z. B. viele Rechnungen, aber wenige Sonderformulare). Für robuste Evaluationen sind daher stratifizierte Splits und klassenbewusste Kennzahlen wichtig; große Referenzdatensätze wie RVL-CDIP zeigen zugleich, dass breite Klassenabdeckung ein wesentlicher Faktor für stabile Modellvergleiche ist. [16] [23]

Besondere Aufmerksamkeit erfordert Data Leakage: Wenn inhaltlich sehr ähnliche Dokumente oder Seiten desselben Falls versehentlich in verschiedenen Splits landen, werden Ergebnisse künstlich verbessert. Methodisch lässt sich das über gruppenbasierte Aufteilungen (z. B. GroupKFold/GroupShuffleSplit) vermeiden, bei denen zusammengehörige Dokumente nur einem Split zugeordnet werden. [24]

3.1.5.4. Best Practices für die Umsetzung

Für die praktische Umsetzung hat sich ein iteratives Vorgehen bewährt: Zunächst wird ein reproduzierbarer Baseline-Preprocessing-Stand definiert, danach werden einzelne Verarbeitungsschritte (z. B. Deskewing, Binarisierung, Layout-Normalisierung) kontrolliert variiert und ihr Einfluss auf Qualitätsmetriken gemessen. So wird sichtbar, welche Maßnahmen tatsächlich Modellnutzen erzeugen und welche lediglich zusätzliche Komplexität verursachen. [20] [22]

Zusammenfassend ist Datenaufbereitung in Dokumentprojekten als eigenständige Engineering-Aufgabe zu behandeln. Je systematischer Preprocessing, Annotation und Evaluationsdesign umgesetzt werden, desto zuverlässiger lassen sich Dokumenttypen automatisiert erkennen und in stabile Geschäftsprozesse integrieren. [2] [24]

3.2. Architekturen für Dokumentensysteme

3.2.1. Suche und Filter-Architekturen

3.2.1.1. Suchalgorithmen im Information Retrieval

Suche und Filter bilden in Dokumentensystemen eine gemeinsame Architekturaufgabe: Die Suche identifiziert relevante Kandidaten, während Filter die Ergebnisliste auf fachlich gültige Teilmengen einschränken. Im Information Retrieval (IR) hat sich dafür eine mehrstufige Verarbeitung etabliert, die Abfrageinterpretation, Kandidatenfindung, Ranking und nachgelagerte Einschränkungen kombiniert. [25] [26]

Für unternehmensweite Dokumentplattformen ist diese Trennung besonders wichtig, weil dieselbe Anfrage häufig sowohl unstrukturierte Textfelder als auch strukturierte Metadaten (z. B. Dokumenttyp, Zeitraum, Abteilung) adressiert. Eine belastbare Sucharchitektur muss daher Ranking- und Filterlogik technisch klar trennen, aber in einer gemeinsamen Query-Pipeline ausführbar machen. [26]

3.2.1.2. Lexikalische Suchverfahren (Keyword Search)

Die lexikalische Suche ist der klassische IR-Ansatz und basiert auf dem Abgleich von Termen zwischen Query und Dokumentbestand. Sie ist besonders stark bei exakten Treffern, etwa bei IDs, normierten Begriffen, Produktcodes oder eindeutig benannten Entitäten. [25]

Inverted Index und TF-IDF

Die technische Basis lexikalischer Systeme ist der Inverted Index: Für jeden Term wird eine Postings-Liste der zugehörigen Dokumente geführt, sodass Anfragen nicht über alle Dokumente sequentiell laufen müssen. Diese Datenstruktur ist in IR-Systemen der Standard, weil sie Suchanfragen in großen Korpora effizient auswertbar macht. [25]

Auf Ranking-Ebene wird häufig mit termbasierten Gewichtungen gearbeitet. Ein klassisches Schema ist TF-IDF, bei dem ein Term höher gewichtet wird, wenn er in einem Dokument häufig vorkommt, aber im Gesamtkorpus vergleichsweise selten ist. Dadurch erhalten diskriminierende Begriffe mehr Einfluss auf die Relevanzbewertung als sehr häufige, wenig informative Terme. [27]

BM25

In praktischen Suchsystemen ist BM25 der verbreitete Standard für lexikalisches Ranking. Das Modell erweitert den reinen TF-IDF-Gedanken um Termfrequenz-Sättigung und Längennormalisierung und reduziert damit typische Verzerrungen zugunsten sehr langer Dokumente. [28]

Dass BM25 nicht nur theoretisch, sondern auch produktiv relevant ist, zeigt sich in verbreiteten Suchbibliotheken: Lucene führt BM25 als zentrale Similarity-Variante und erlaubt zusätzlich die Parametrisierung über k_1 und b für unterschiedliche Korpuscharakteristika. [29]

3.2.1.3. Semantische Suchverfahren (Vector Search)

Semantische Suchverfahren adressieren eine zentrale Schwäche rein lexikalischer Verfahren: relevante Treffer enthalten häufig nicht exakt dieselben Terme wie die Suchanfrage, sind aber inhaltlich nahe verwandt. Daher werden Query und Dokumente in einen gemeinsamen Vektorraum überführt und über Distanz- bzw. Ähnlichkeitsmaße verglichen. [30]

Embeddings werden in dieser Arbeit bereits in Abschnitt 3.1.3 eingeführt und dort detailliert erklärt. Für die Sucharchitektur ist hier nur relevant: Die Qualität der semantischen Suche hängt davon ab, dass Query- und Dokumentvektoren mit kompatiblen Modellen und identischen Dimensionsannahmen erzeugt werden. [30]

Suche mittels k-NN und HNSW

Das grundlegende Suchproblem in Vektorräumen ist k-NN (k-nearest neighbors): Gesucht werden die k nächsten Dokumentvektoren zur Query. Da exakte k-NN-Suche bei großen Datenmengen teuer wird, setzen produktive Systeme meist auf ANN-Methoden (Approximate Nearest Neighbor), die Rechenzeit gegen eine kontrollierte Recall-Abweichung tauschen. [30]

Ein verbreiteter ANN-Ansatz ist HNSW (Hierarchical Navigable Small World). Der Algorithmus organisiert Vektoren in einer hierarchischen Navigationsstruktur und erreicht dadurch hohe Suchgeschwindigkeit bei gleichzeitig guter Trefferqualität, was ihn für große Dokumentindizes praktikabel macht. [31]

3.2.1.4. Hybride Suchansätze (Hybrid Search)

In realen Dokumentanwendungen deckt weder ein rein lexikalischer noch ein rein semantischer Retriever alle Abfragetypen optimal ab. Hybrid Search kombiniert deshalb beide Signale in einer gemeinsamen Retrieval-Pipeline und erhöht so die Robustheit gegenüber unterschiedlichen Query-Formen. [30]

Eine verbreitete Fusionsstrategie ist Reciprocal Rank Fusion (RRF). RRF kombiniert mehrere Ergebnislisten rein rangbasiert und benötigt keine aufwändige Score-Normalisierung zwischen unterschiedlichen Retrievalmethoden, was die praktische Integration deutlich vereinfacht. [32]

3.2.1.5. Filter- und Facettenarchitekturen

Filter sind kein Ersatz für Ranking, sondern eine komplementäre Architekturkomponente. In Elasticsearch wird diese Trennung explizit als Query Context (scored) versus Filter Context (binär, nicht-scorebasiert) modelliert; insbesondere 'bool.filter'-Klauseln werden ohne Relevanzbewertung ausgeführt und für wiederholte Nutzung gecacht. [26] [33]

Für fachliche Navigation in großen Trefferlisten werden zusätzlich Facetten bzw. Aggregationen eingesetzt. Typische Muster sind Terms-Buckets (z. B. Dokumenttyp, Abteilung) und zeitliche Buckets (Date Histogram), die eine schrittweise Eingrenzung der Suchergebnisse ermöglichen. [34] [35] [36]

Dass diese Logik plattformübergreifend etabliert ist, zeigt auch der Solr-Ansatz: Faceting wird dort als Kategorisierung von Suchresultaten über indizierte Terme beschrieben und als zentrales Interaktionsmuster für explorative Suche eingesetzt. [37]

Für die Systemarchitektur ergibt sich daraus ein stabiler Aufbau mit klaren Verantwortlichkeiten:

1. **Retriever-Schicht** für lexikalische und/oder semantische Kandidatensuche, [30]
2. **Ranking-/Fusion-Schicht** für die Ergebniszusammenführung (z. B. RRF), [32]
3. **Filter-/Facet-Schicht** für fachliche Constraints und UI-gestützte Eingrenzung. [26] [35]

Diese Schichtung verbessert Wartbarkeit und Skalierbarkeit, weil Relevanzmodelle, Filterregeln und Navigationslogik unabhängig weiterentwickelt werden können. Gerade in Dokumentensystemen mit häufig wechselnden Metadatenanforderungen ist diese Entkopplung ein zentraler Qualitätsfaktor. [26]

3.2.1.6. Vergleich der Verfahren

Die folgende Tabelle 3.6 stellt die in der Praxis wichtigsten Unterschiede zwischen lexikalischer, semantischer und hybrider Suche gegenüber.

3.2.2. Rollen- und Berechtigungssysteme

3.2.2.1. Aufbau einer Berechtigung

Eine Berechtigung setzt sich grundsätzlich aus zwei zentralen Komponenten zusammen, die gemeinsam definieren, welcher Zugriff in einem System erlaubt oder untersagt ist.

[38]

3.2.2.2. Arten von Berechtigungen

Berechtigungskonzepte bilden eine wesentliche Grundlage für die Zugriffskontrolle in IT-Systemen. Die elementarste Form stellt dabei die *binäre Berechtigung* dar. Sie beschreibt

Kriterium	Lexikalische Suche	Semantische Suche	Hybride Suche
Primäres Signal	Termübereinstimmung (z. B. BM25)	Vektorähnlichkeit (k-NN/ANN)	Kombination beider Signale
Stärken	Exakte Treffer bei IDs und Fachbegriffen	Robust bei Synonymen und paraphrasierten Anfragen	Höhere Robustheit über gemischte Query-Typen
Grenzen	Schwach bei semantischer Varianz	Abhängig von Embedding-Qualität und Indexparametern	Höherer Integrations- und Betriebsaufwand
Filterintegration	Sehr gut mit Metadatenfiltern kombinierbar	Ebenfalls kombinierbar, teils mit speziellen ANN-Strategien	Standardmuster: Ranking + Filter + Facets

Tabelle 3.6.: Vergleich von Suchansätzen in Dokumentsystemen [28] [30] [32]

Komponente	Beschreibung
Zu schützende Ressource	Beschreibt das Objekt, auf das sich eine Berechtigung bezieht. Dabei kann zwischen funktionalen und inhaltsbezogenen Ressourcen unterschieden werden. Die Definition der Ressource kann auf unterschiedlichen Granularitätsebenen erfolgen, beispielsweise global für ein gesamtes System oder atomar für einzelne Systembestandteile.
Zugehörige Operation auf der Ressource	Legt fest, welche konkrete Handlung in Bezug auf die definierte Ressource durch eine Berechtigung erlaubt oder eingeschränkt wird. Die Operation bestimmt, welche Aktionen eine Identität auf der jeweiligen Ressource ausführen darf.

Tabelle 3.7.: Komponenten einer Berechtigung

eine eindeutig definierte Zugriffserlaubnis, die sich jeweils auf ein konkretes Objekt und eine einzelne Operation bezieht. Der Zugriff ist entweder erlaubt oder verweigert, wodurch diese Berechtigungsform ausschließlich zwei mögliche Zustände kennt. [38]

Aufgrund dieser klaren Struktur eignen sich binäre Berechtigungen besonders für die technische Überprüfung von Zugriffsrechten. Umfangreichere Berechtigungsmodelle, die mehrere Objekte oder Operationen umfassen, werden vor der eigentlichen Zugriffskontrolle in einzelne binäre Berechtigungen zerlegt. Die endgültige Zugriffsentscheidung erfolgt anschließend ausschließlich auf Basis dieser elementaren Berechtigungseinheiten. [38]

Dieses Vorgehen findet auch bei weiterführenden Steuerungsmechanismen Anwendung, beispielsweise bei der Verwendung von Rollen oder Profilen. Solche abstrakten Konzepte werden zunächst auf konkrete Berechtigungen abgebildet und anschließend in mehrere binäre Einzelrechte aufgelöst, die festlegen, ob eine bestimmte Handlung erlaubt oder untersagt ist.

[38]

Neben binären Berechtigungen existieren sogenannte *Wertberechtigungen*, bei denen mehrere Einzelrechte in einem numerischen Wert zusammengefasst werden. Ein verbreitetes Anwendungsbeispiel hierfür ist das Berechtigungssystem von UNIX-Dateisystemen. Dort repräsentiert ein Zahlenwert die Kombination einzelner Zugriffsrechte wie Lesen, Schreiben und Ausführen. Der Wert 7 steht für den vollständigen Zugriff und ergibt sich aus der Addition der einzelnen Rechtewerte. [38]

Wertberechtigungen stellen somit eine kompakte Darstellungsform mehrerer binärer Berechtigungen dar. Obwohl sie die Verwaltung von Zugriffsrechten vereinfachen, basiert die tatsächliche Zugriffskontrolle weiterhin auf der Auswertung der zugrunde liegenden binären Berechtigungen. [38]

3.2.2.3. Scope von Berechtigungen

Ein zentrales Kriterium bei der Konzeption und Platzierung von Berechtigungen ist die Festlegung, welche Zugriffsrechte durch eine Berechtigung gewährt werden sollen. In diesem Zusammenhang kommt dem Grad der Differenzierung von Berechtigungen, der als Granularität bezeichnet wird, eine besondere Bedeutung zu. [38]

Die Grundlage für die Ausgestaltung von Berechtigungen bildet der Schutzbedarf der jeweiligen Ressource. Dieser wird üblicherweise im Rahmen einer Schutzbedarfsanalyse ermittelt, bei der unterschiedliche Einflussfaktoren berücksichtigt werden. Dazu zählen unter anderem die Relevanz der Ressource für Geschäftsprozesse sowie die Kritikalität ihrer Funktionen und Inhalte. [38]

Die ermittelten Schutzanforderungen fließen anschließend in die Berechtigungsplanung ein. Dabei wird festgelegt, auf welcher Detailebene der Zugriffsschutz umgesetzt werden muss und welche Funktionen durch entsprechende Berechtigungen abzusichern sind. Darüber hinaus wird bestimmt, wie stark der Zugriffsschutz ausgeprägt sein soll, beispielsweise ob der Einsatz erweiterter Authentifizierungsmechanismen erforderlich ist. [38]

3.2.2.4. Berechtigungsstufen

Anstatt jedem Nutzer komplizierte Einzelrechte zu geben, werden Berechtigungen in feste Stufen eingeteilt. Das macht die Verwaltung einfach: Man muss einem Nutzer nur noch eine **Ziffer** (die Stufe) zuweisen. [38] Die Skala reicht von *keine Berechtigung* bis *alle Berechtigungen*.

- **Stufe 0: Kein Zugriff**
  (Gar keine Rechte)
- **Stufe 1: Nur lesen**
  (Informationen ansehen)
- **Stufe 2: Lesen und Hinzufügen**
  (Daten ansehen und neue erstellen)

- **Stufe 3: Ändern und Löschen**
  (Bestehende Daten bearbeiten)
- **Stufe 4: Vollzugriff**
  (Alle Rechte)

3.2.3. Skalierbare plattformunabhängige Systemarchitekturen

3.2.3.1. Skalierbarkeit

Unter einer skalierbaren Systemarchitektur versteht man ein System, das sich problemlos an steigende Anforderungen anpassen lässt. Fehlende Skalierbarkeit führt dazu, dass Systeme bei wachsender Nutzerzahl oder zunehmendem Datenvolumen spürbare Performanceprobleme entwickeln. Während des Entwicklungsprozess sind unterschiedliche Prinzipien zu beachten:

Merkmal	**Beschreibung**
Modularität	Einzelne Komponenten der Software sollen unabhängig sein und sich flexibel aktualisieren oder erweitern lassen.
Flexibilität	Systeme müssen sich dynamisch an verändernde Anforderungen anpassen können.
Fehlertoleranz	Das System soll auf Fehler reagieren können und automatisch geeignete Maßnahmen ergreifen.

Tabelle 3.8.: Merkmale von Softwaresystemen

[39]

Um eine skalierbare Architektur zu gewährleisten, sind Optimierungsfunktionen von Bedeutung, um die Effizienz und Leistung zu optimieren. Wichtige Optimierungsstrategien sind unter anderem:

Technik	**Beschreibung**
Load Balancing	Verteilung von Anfragen auf mehrere Server, um Überlastung zu vermeiden.
Caching	Speicherung häufig abgerufener Daten, um Zugriffszeiten zu verkürzen.
Partitioning	Aufteilung von Daten in kleinere Einheiten, die parallel verarbeitet werden können.
Asynchrone Verarbeitung	Ermöglicht es Systemen, Aufgaben im Hintergrund auszuführen, was die Reaktionszeit verbessert.

Tabelle 3.9.: Techniken zur Leistungsoptimierung

[39]

Das Hauptziel einer skalierbaren Architektur besteht darin, auch bei steigender Nutzerzahl und wachsendem Datenvolumen eine hohe Verfügbarkeit sowie eine konsistent hohe Leistung

sicherzustellen. Gleichzeitig muss das System so gestaltet sein, dass es leicht wartbar und problemlos erweiterbar bleibt, um zukünftige Anforderungen effizient integrieren zu können. Insgesamt ist eine durchdacht geplante skalierbare Architektur ein entscheidender Faktor für die langfristige Stabilität, Flexibilität und den Erfolg moderner Softwaresysteme. [39]

3.2.3.2. Plattformunabhängigkeit

Ein Computerprogramm benötigt eine Umgebung, in der es gestartet werden kann und während der gesamten Laufzeit stabil funktioniert. Ein Programm gilt als plattformunabhängig oder plattformübergreifend, wenn es auf verschiedenen Computersystemen ausgeführt werden kann, also auf Geräten mit unterschiedlicher Hardware, verschiedenen Prozessoren oder unterschiedlichen Betriebssystemen. Der Grad dieser Unabhängigkeit wird als Portierbarkeit (oder Portabilität) bezeichnet.

Die Portabilität kann z. B. geschätzt werden über

$$P = 1 - \frac{U + A}{E}$$

mit

- Übertragungsaufwand U (insbesondere Neukompilierung),
- Anpassungsaufwand A (Änderung des Quellcodes, z. B. bei Austausch von Betriebsschnittstellen),
- Entwicklungsaufwand E für Neuentwicklung.

Eine Portabilität von $P = 1$ bedeutet vollständige Kompatibilität; das Programm ist also ohne Änderungen auf dem Zielsystem lauffähig, was genau dann gilt, wenn $U = A = 0$.

Eine Quellcode-Portabilität liegt im Regelfall vor, wenn die Gesamt-Portabilität über 90% liegt. Dies entspricht einem Anpassungsaufwand von $A = 0$ und einem Übertragungsaufwand von $U < 0{,}1E$, da bei

$$P = 1 - \frac{U}{E} > 0{,}9$$

der Wert für U kleiner als ein Zehntel von E sein muss. Eine Portabilität nahe 0 entspricht hingegen einer nahezu vollständigen Neuentwicklung des Programms, wobei in diesem Fall $P \approx 0$ und somit $U + A \approx E$ gilt. [40]

Portabilität ist kein Maß für die Lauffähigkeit eines Programms auf der Zielplattform, d. h. selbst eine Portabilität von 99 % bedeutet nicht unbedingt, dass das Programm nutzbar ist, sondern lediglich, dass eine Portierung im Vergleich zu einer Neuentwicklung deutlich weniger Aufwand erfordert. Damit ist nicht nur gemeint, dass ein Programm auf mehreren Plattformen laufen kann, sondern auch, wie viel Aufwand nötig ist, um es dafür anzupassen. Dieser Vorgang wird Portierung oder Migration genannt. [40]

3.2.4. Sharepoint

3.2.4.1. Einordnung und Produkte

Microsoft SharePoint fungiert als webbasierte Kollaborationsplattform, die primär auf die Optimierung der teaminternen Zusammenarbeit sowie ein effizientes Informationsmanagement ausgelegt ist. Durch die Bereitstellung zentraler Instanzen zur Inhaltsverwaltung ermöglicht die Plattform eine strukturierte Ablage und Verteilung von Daten. Ziel ist es, durch zusammenwirkende Kommunikation und medienbruchfreie Kooperation die organisatorische Produktivität zu steigern. [41]

Produkt	**Kernmerkmale und Fokus**
SharePoint in Microsoft 365	Cloudbasierte Plattform (SaaS) zur zentralen Inhaltsverwaltung und teamübergreifenden Freigabe von Dokumenten. Nutzung erfolgt im Rahmen von Microsoft-365-Abonnements.
SharePoint Server	On-Premise-Lösung für den Betrieb auf eigener IT-Infrastruktur. Ermöglicht volle Kontrolle über Datenhoheit und unterstützt hybride Szenarien mit der Cloud.
SharePoint Designer 2013	Spezialisiertes Werkzeug zur Erstellung deklarativer Workflows und zur Modellierung komplexer Geschäftsprozesse innerhalb der SharePoint-Umgebung.
OneDrive	Schnittstelle zur Synchronisation von SharePoint-Bibliotheken mit lokalen Endgeräten. Ermöglicht die Offline-Bearbeitung und Dateiverwaltung im lokalen Dateisystem.

Tabelle 3.10.: Überblick über SharePoint-Produkte

[41]

3.2.4.2. Architektur und Hierarchie

Die Struktur von SharePoint ist hierarchisch aufgebaut, um Inhalte logisch zu trennen und den Zugriff gezielt zu steuern. Den Kern dieser Architektur bildet die sogenannte Websitesammlung (Site Collection). [42]

Eine Websitesammlung besteht immer aus einer Website auf der obersten Ebene und allen darunterliegenden Websites. Sie dient als übergeordneter Container, in dem zentrale Einstellungen für die Sicherheit, das Design und verschiedene Funktionen festgelegt werden. Technisch gesehen landen alle Daten einer solchen Sammlung in einer einzigen Inhaltsdatenbank. Zwar kann eine Datenbank mehrere Websitesammlungen speichern, aber eine Sammlung kann nicht über mehrere Datenbanken verteilt werden. [42]

Bei der Planung der Struktur gibt Microsoft die Empfehlung ab, für jede Arbeitseinheit eine eigene Websitesammlung anzulegen, anstatt eine komplexe Verschachtelung mit vielen

Unterwebsites zu bauen. Das hat den Vorteil, dass die Umgebung übersichtlich bleibt und später einfacher migriert werden kann. [42]

Innerhalb einer solchen Sammlung können Ressourcen wie Bilder, Vorlagen oder bestimmte Spaltentypen gemeinsam genutzt werden. Das sorgt dafür, dass die Navigation und das Erscheinungsbild für die Nutzer einheitlich bleiben. Bei der Benennung der URLs wird meistens auf pfadbasierte Adressen (wie z. B. „/sites/Projektname“) gesetzt, da diese mit Tools wie der PowerShell am einfachsten zu verwalten sind. [42]

3.2.4.3. Funktionale Elemente einer SharePoint-Website

Innerhalb einer Websitesammlung bestehen die einzelnen Websites aus verschiedenen funktionalen Elementen, die für die Organisation und Verwaltung der Daten zuständig sind:

- **Dokumentbibliotheken (Document Libraries):** Diese stellen den primären Speicherort für Dateien dar. Im Gegensatz zu herkömmlichen Dateiordnern ermöglichen sie die Nutzung von erweiterten Funktionen wie Versionierung, Check-Out-Mechanismen und die Verwendung von Metadaten. [43]
- **Listen (Lists):** Listen dienen der Erfassung strukturierter Daten, ähnlich einer Tabelle. Sie werden beispielsweise für Aufgabenlisten, Kontaktverzeichnisse oder Inventarlisten verwendet. [43]
- **Seiten (Pages):** Diese bilden das visuelle Interface der Website. Informationen werden hier mithilfe von sogenannten Webparts (funktionalen Bausteinen) für den Endanwender aufbereitet. [43]
- **Metadaten (Spalten):** Anstatt Dateien ausschließlich in starren Ordnerstrukturen zu sortieren, erlauben Spalten das Hinzufügen von Attributen wie „Dokumententyp“ oder „Status“. Dies verbessert die Filterbarkeit und Auffindbarkeit der Dokumente innerhalb der gemeinsamen Ablage erheblich. [43]

3.2.4.4. Kernfunktionen der Dokumentenverwaltung

SharePoint bietet eine Vielzahl von Funktionen, die speziell auf die Anforderungen der Dokumentenverwaltung zugeschnitten sind:

- **Versionierung:** Jede Änderung an einem Dokument wird als separate Version gespeichert. Dadurch können Nutzer frühere Versionen wiederherstellen oder Änderungen nachverfolgen. [43]
- **Auschecken und Einchecken (Check-Out/In):** Um Bearbeitungskonflikte zu vermeiden, können Dokumente exklusiv für einen Nutzer gesperrt werden. Während eine Datei ausgecheckt ist, können andere Nutzer diese zwar lesen, aber keine Änderungen vornehmen, bis sie wieder eingecheckt wird. [43]
- **Gemeinsame Dokumenterstellung (Co-Authoring):** Diese Funktion erlaubt es mehreren Anwendern, zeitgleich an demselben Dokument (z. B. Word oder Excel) zu arbeiten. Änderungen werden in Echtzeit synchronisiert, was die kollaborative Zusammenarbeit beschleunigt. [43]

3.2.5. Microsoft Entra ID

3.2.5.1. Begriff und Einordnung

Microsoft Entra beschreibt eine umfassende Produktfamilie für Identitätsmanagement und Netzwerkzugriff, die darauf ausgelegt ist, eine moderne Zero Trust-Sicherheitsstrategie in Organisationen zu etablieren. Das Ziel dieser Architektur ist die Schaffung einer Vertrauensstruktur, die Identitäten sowie Zugriffsbedingungen und Berechtigungen konsequent überprüft, Verbindungskanäle verschlüsselt und eine kontinuierliche Überwachung auf Kompromittierungen ermöglicht. Die hierarchische Struktur und die verschiedenen Säulen dieser Architektur sind in Abbildung 3.2 dargestellt. [44]

Das zentrale Kernprodukt dieser Familie ist Microsoft Entra ID. Dabei handelt es sich um einen cloudbasierten Dienst zur Identitäts- und Zugriffsverwaltung, der die grundlegenden Funktionen für Authentifizierung und den Schutz von Benutzern, Geräten sowie Anwendungen bereitstellt. Eine wesentliche Besonderheit für Unternehmen im Microsoft-Umfeld besteht darin, dass Abonnenten von Diensten wie Microsoft 365 oder Azure automatisch einen Microsoft Entra-Mandanten nutzen und somit direkt mit der Verwaltung ihrer Cloud-Anwendungen beginnen können. Über das webbasierte Microsoft Entra Admin Center lassen sich diese Produkte zentral über eine einzige Benutzeroberfläche konfigurieren und verwalten. [44]

Abbildung 3.2.: Übersicht der Microsoft Entra Produktfamilie [44]

3.2.5.2. Zentrale Identitätsverwaltung

Die zentrale Verwaltung innerhalb von Microsoft Entra sorgt dafür, dass Identitäten für unterschiedliche Nutzergruppen und Ressourcen an einem einzigen Ort kontrolliert werden. Dies umfasst nicht nur menschliche Identitäten wie Mitarbeitende, Partner oder Kundschaft, sondern schließt auch Geräte sowie sogenannte Workload-Identitäten (z. B. Anwendungen und Dienste) mit ein. [44]

Ein wesentlicher Bestandteil dieser Verwaltung ist die Microsoft Entra ID Governance. Diese ermöglicht es, den gesamten Lebenszyklus einer Identität zu automatisieren. So können beispielsweise Zugriffsanfragen, Zuweisungen von Lizenzen und regelmäßige Überprüfungen von Berechtigungen systemgesteuert erfolgen. Dies stellt sicher, dass neuen Mitarbeitern automatisch die benötigten Ressourcen zugewiesen werden und diese Zugänge beim Verlassen des Unternehmens ebenso zuverlässig wieder entfernt werden. [44]

Für die Verwaltung externer Nutzer bietet Microsoft Entra External ID eine sichere Methode zur Zusammenarbeit. Hiermit können Geschäftspartner oder Gäste gezielt Zugriff auf interne Ressourcen erhalten, während Kunden über Self-Service-Registrierungen (z. B. via Google- oder Facebook-Konten) angebunden werden können. [44]

3.2.5.3. Authentifizierung und Autorisierung

Innerhalb von Microsoft Entra ID bilden Authentifizierung und Autorisierung die Grundlage für den sicheren Zugriff auf Ressourcen, folgen jedoch unterschiedlichen Prinzipien:

- **Authentifizierung (Identity):** Dies ist der Prozess der Identitätsprüfung. Hierbei stellt das System sicher, dass ein Benutzer tatsächlich die Person ist, für die er sich ausgibt. Dies geschieht in der Regel durch die Abfrage von Anmeldedaten oder sicheren Methoden wie der Multi-Faktor-Authentifizierung (MFA). Microsoft Entra ID Protection unterstützt diesen Vorgang, indem es risikobasierte Richtlinien nutzt, um verdächtige Anmeldeversuche automatisch zu erkennen und zusätzliche Bestätigungen einzufordern. [44]
- **Autorisierung (Permissions):** Nach einer erfolgreichen Anmeldung bestimmt die Autorisierung, welche spezifischen Zugriffsrechte der Benutzer besitzt. Es wird festgelegt, auf welche Anwendungen, Dateien oder Funktionen (z. B. innerhalb einer SharePoint-Website) zugegriffen werden darf. [44]

3.2.5.4. Single Sign-On (SSO)

Ein zentrales Leistungsmerkmal von Microsoft Entra ID ist das sogenannte Single Sign-On. Diese Funktion ermöglicht es Benutzern, sich mit nur einer einzigen Identität – bestehend aus einem Benutzernamen und einem Kennwort – einmalig anzumelden, um danach automatisch Zugriff auf alle für sie freigegebenen Cloud-Anwendungen sowie lokalen Ressourcen zu erhalten. [44]

Aus technischer Sicht bietet SSO zwei wesentliche Vorteile für das Unternehmen:

- **Benutzerfreundlichkeit:** Da sich die Anwender nicht mehr für jede einzelne Anwendung (wie SharePoint, Teams oder externe Drittanbieter-Tools) unterschiedliche Zugangsdaten merken müssen, wird die tägliche Arbeit effizienter. Zudem sinkt die Anzahl an Support-Anfragen aufgrund vergessener Passwörter erheblich. [44]
- **Sicherheit:** Da die Identität zentral an einer Stelle geprüft wird, können Sicherheitsrichtlinien, wie beispielsweise die Mehrfaktor-Authentifizierung (MFA), global für alle angebundenen Programme erzwungen werden. Zudem wird das Risiko minimiert, dass

Nutzer unsichere Passwörter für mehrere Dienste gleichzeitig verwenden oder diese ungesichert notieren. [44]

3.2.5.5. Integration externer Anwendungen

Ein wesentlicher Vorteil von Microsoft Entra ID ist die Fähigkeit, über die Grenzen der Microsoft-Welt hinaus als zentrale Identitätsinstanz zu fungieren. Dies wird durch die Integration externer Anwendungen (Third-Party-Apps) realisiert:

- **SaaS-Anwendungen:** Über vorkonfigurierte Schnittstellen lassen sich gängige Cloud-Dienste (wie Salesforce, Slack oder Dropbox) nahtlos anbinden. Dadurch profitieren auch diese externen Dienste von Sicherheitsfeatures wie Single Sign-On (SSO) und der Mehrfaktor-Authentifizierung. [44]
- **Zentrale App-Verwaltung:** Administratoren können im Entra Admin Center exakt steuern, welche Benutzer oder Gruppen Zugriff auf bestimmte externe Software erhalten. Dies vereinfacht das Onboarding neuer Mitarbeiter, da Zugänge für verschiedene Plattformen an einer zentralen Stelle zugewiesen werden können. [44]
- **Sicherer Internet- und Privatzugriff:** Moderne Komponenten wie der *Microsoft Entra Internet Access* und *Private Access* erweitern diesen Schutz. Sie ermöglichen einen sicheren Zugriff auf Internet-Ressourcen sowie interne Unternehmensnetzwerke, ohne dass klassische, oft unsicherere VPN-Verbindungen notwendig sind. [44]

Die zentrale Rolle von Microsoft Entra ID als Vermittler zwischen lokalen Infrastrukturen, Cloud-Applikationen und externen Identitäten wird in Abbildung 3.3 verdeutlicht. Durch diese weitreichende Integrationsfähigkeit wird Microsoft Entra ID zum zentralen Kontrollpunkt für die gesamte IT-Infrastruktur einer Organisation, was die Sicherheit erhöht und die Komplexität für die Endanwender reduziert.

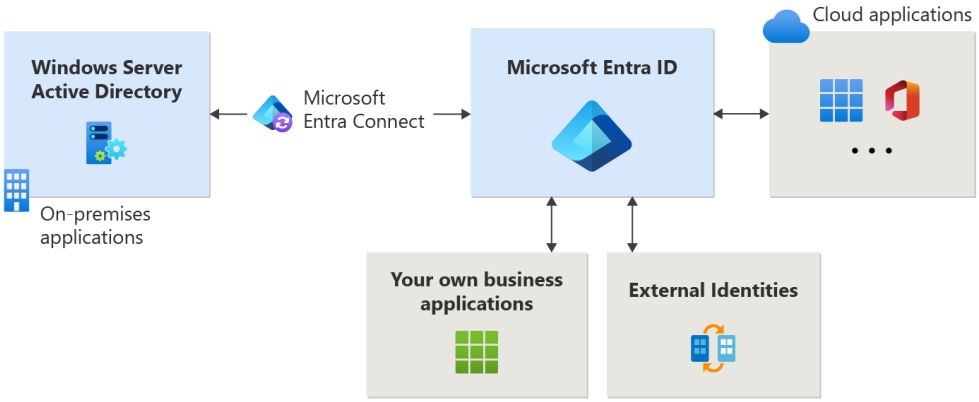

Abbildung 3.3.: Integration von On-Premises- und Cloud-Anwendungen über Microsoft Entra ID [44]

4. Dokumentation der Implementierung

4.1. Dokumentation - Grundlegend

4.1.1. Test Umgebung

4.1.2. Technologien

4.2. Dokumentation - Funktionen

4.2.1. Dokumenten-Upload

Dieses Kapitel dokumentiert den vollständigen Upload-Flow von der Anlage einer Upload-Zone bis zur finalen Speicherung des Dokuments in SharePoint inklusive Metadatenpersistenz und Suchindex-Aktualisierung.

4.2.1.1. Technischer Gesamtflow

Der Upload ist als mehrstufige Pipeline umgesetzt, damit fachliche Prüfung, Benutzerinteraktion und Persistenz klar getrennt bleiben:

1. Eine Zone wird im Frontend erstellt oder ausgewählt.
2. Die Zone wird im Zone-Service persistiert; der zugehörige SharePoint-Ordner wird best-effort angelegt.
3. Eine Datei wird per Dateiauswahl oder Drag & Drop in die Zone-Upload-Queue gestellt.
4. Der Zone-Service führt eine Preflight-Phase aus und ruft die Dokumenten-Klassifikation auf.
5. Ergebniswerte der Klassifikation werden zusammen mit der Datei temporär abgelegt und mit `uploadId` referenziert.
6. Die Benutzerin bzw. der Benutzer bestätigt oder korrigiert die erkannten Metadaten.
7. Beim `commit` wird die Datei nach SharePoint geschrieben und die temporäre Ablage aufgelöst.
8. Die Dokument-Metadaten werden in der Documents-API in der Datenbank gespeichert.
9. Das Dokument wird für die Suche über die Search-API indiziert.

Damit kann der Uploadprozess robust aufgeteilt werden: Die KI-Extraktion ist vor der finalen Persistenz sichtbar und korrigierbar, bevor irreversible Schreiboperationen durchgeführt werden.

4.2.1.2. Zone-Erstellung und SharePoint-Ordnerinitialisierung

Der Upload beginnt fachlich mit einer Zone. Die Zone-API stellt dafür `POST /zones` bereit. Beim Anlegen wird ein neuer `ZoneDto`-Eintrag in der zonenspezifischen JSON-Struktur gespeichert und parallel versucht, den Zielordner in SharePoint zu erzeugen.

Quellcode 4.1: Zone-Erstellung mit SharePoint-Ordnerinitialisierung

```
g.MapPost("", async (ZoneDto zone, IZonesStore store, IUserContext user,
                     ITokenAcquisition tokenAcquisition,
                     IHttpClientFactory httpFactory, IConfiguration cfg,
                     CancellationToken ct) =>
{
    var folderName = zone.FolderPath.Trim('/').Split('/').Last();
    var driveId = cfg["SharePoint:DriveId"] ?? zone.DriveId ?? "";

    var doc = await store.UpsertAsync(user.TenantId, user.ObjectId, d => {
        var list = d.Zones.ToList();
        var newZone = zone with { Id = Guid.NewGuid(), OrderIndex = list.Count + 1, DriveId = driveId };
        list.Add(newZone);
        return d with { Zones = list };
    }, null, ct);

    try {
        var token = await tokenAcquisition.GetAccessTokenForUserAsync(
            new[] { "https://graph.microsoft.com/Files.ReadWrite.All" });
        var client = httpFactory.CreateClient("graph");
        client.DefaultRequestHeaders.Authorization = new AuthenticationHeaderValue("Bearer", token);
        var payload = new { name = folderName, folder = new { }, microsoft_graph_conflictBehavior = "fail" };
        await client.PostAsJsonAsync($"drives/{driveId}/root/children", payload, ct);
    } catch {}

    return Results.Ok(doc);
});
```

Die Ordneranlage ist absichtlich tolerant implementiert (`try/catch` ohne Abbruch), sodass die fachliche Zone nicht verloren geht, wenn die Graph-Operation nicht unmittelbar erfolgreich ist.

4.2.1.3. Frontend-Uploadsteuerung über UploadQueueService

In der Webanwendung wird der Upload nicht direkt aus der Seite abgesendet, sondern über den `UploadQueueService` sequenziell verarbeitet. Dadurch können mehrere Dateien nacheinander pro Zone verarbeitet werden, inklusive Statusmeldungen und Dialogsteuerung.

Quellcode 4.2: Preflight- und Commit-Steuerung im UploadQueueService

```
private async Task ProcessRequestAsync(UploadRequest request,
    CancellationToken ct)
{
    if (request.SizeBytes > MaxUploadBytes) return; // 100 MB Grenze

    using var form = new MultipartFormDataContent();
    form.Add(new StreamContent(await OpenUploadStreamAsync(request, ct)),
             "file", request.FileName);

    var res = await client.PostAsync($"zones/{request.ZoneId}/upload/
    preflight", form, ct);
    if (!res.IsSuccessStatusCode) return;

    var preflight = await res.Content.ReadFromJsonAsync<UploadPreflight>(
    cancellationToken: ct);
    CurrentPreflight = preflight;
    _decision = new TaskCompletionSource<UploadDecision>(
    TaskCreationOptions.RunContinuationsAsynchronously);

    var decision = await _decision.Task.WaitAsync(ct);
    if (!decision.Confirmed || decision.Payload is null) return;

    await CommitAsync(client, decision.Payload, ct);
}
```

Wesentliche technische Punkte in dieser Schicht sind:

- zentrale Dateigrößenbegrenzung auf 100 MB,
- einheitlicher Status pro Zone (z. B. `"KI analysiert Dokument..."`),
- ein expliziter Bestätigungszustand zwischen Preflight und Commit,
- sowie identischer Ablauf für Dateiauswahl und Drag-&-Drop.

4.2.1.4. Preflight-Endpunkt und Klassifikationsaufruf

Die eigentliche serverseitige Vorverarbeitung startet über den Preflight-Endpunkt der Zone-API. Der Endpunkt nimmt das Multipart-File entgegen, sendet es an den AI-Service und legt danach Datei plus Metadaten temporär ab.

Quellcode 4.3: Preflight-Endpunkt im Zone-Service

```
g.MapPost("{zoneId:guid}/upload/preflight", async (Guid zoneId,
    HttpRequest req,
                                                FileZonesStore store,
                                                IUserContext user,
                                                IHttpClientFactory
    httpFactory,
                                                CancellationToken ct)
    =>
{
```

```
7       var form = await req.ReadFormAsync(ct);
8       var file = form.Files["file"];
9       if (file == null) return Results.BadRequest("No file");
10
11      var ai = httpFactory.CreateClient("ai");
12      using var mp = new MultipartFormDataContent();
13      mp.Add(new StreamContent(file.OpenReadStream()), "file", file.FileName
        );
14
15      var aiResp = await ai.PostAsync("/ai/classify", mp, ct);
16      if (!aiResp.IsSuccessStatusCode) return Results.Problem("KI-Dienst
        Fehler");
17
18      var extraction = await aiResp.Content.ReadFromJsonAsync<
        DocumentClassificationResultDto>(
19          new JsonSerializerOptions { PropertyNamingPolicy =
        JsonNamingPolicy.CamelCase }, ct);
20
21      var (uploadId, docType) = await store.SaveTempAsync(
22          user.TenantId, user.ObjectId, zoneId, file,
23          extraction!.DocumentType, extraction.DocumentYear,
24          extraction.ContractType, extraction.Partner, extraction.ExpireDate
        , ct);
25
26      return Results.Ok(new {
27          uploadId,
28          documentType = docType.ToString(),
29          year = extraction.DocumentYear,
30          partner = extraction.Partner,
31          contractType = extraction.ContractType,
32          expireDate = extraction.ExpireDate
33      });
34  }).DisableAntiforgery();
```

Die Klassifikation selbst (Prompting, Schema-Bindung, Nachvalidierung) ist in Kapitel 4.2.2 detailliert beschrieben und wird hier nur als aufgerufener Baustein verwendet.

Die Methode `SaveTempAsync(...)` persistiert in dieser Phase:

- die hochgeladene Binärdatei im temporären Bereich,
- ein Metadaten-JSON mit `uploadId`, Zone, Dateieigenschaften und Klassifikation,
- sowie den Zeitstempel der KI-Auswertung (`ClassifiedAtUtc`).

4.2.1.5. Temporäre Ablage und Benutzerkorrektur

Die temporäre Ablage entkoppelt KI-Ergebnis und finale Speicherung. Das Frontend zeigt die zurückgegebenen Werte in einem Dialog (`UploadPreflight`) an. Dort können Dokumenttyp, Jahr, Partner, Vertragsart und Ablaufdatum vor dem Commit angepasst werden.

Die finale Commit-Nachricht enthält damit nicht nur die `uploadId`, sondern den bestätigten Metadatenstand aus der Benutzeroberfläche.

4.2.1.6. Commit in SharePoint und technische Verarbeitung

Der Commit wird über `POST /zones/{zoneId}/upload/{uploadId}/commit` gestartet. Die Hauptarbeit liegt in `CommitTempToSharePointAsync(...)`:

- Laden der temporären Datei und KI-Metadaten,
- Merge mit benutzerbestätigten Änderungen,
- SHA-256-Bildung zur Integritätsreferenz,
- Zielpfad-Aufbau aus Zone-Ordner und optionalem Partner-Unterordner,
- Upload der Datei in SharePoint via Microsoft Graph,
- optionale Textextraktion bei PDF für spätere Suche,
- Löschung der temporären Artefakte.

Quellcode 4.4: Commit-Logik mit SharePoint-Upload

```
var finalMeta = aiMeta with {
    ExpireDate = confirmed.ExpireDate,
    Partner = confirmed.Partner ?? aiMeta.Partner,
    ContractType = confirmed.ContractType ?? aiMeta.ContractType,
    DocumentYear = confirmed.DocumentYear ?? aiMeta.DocumentYear,
    DocumentType = confirmed.DocumentType ?? aiMeta.DocumentType
};

var partnerFolder = SanitizeFolderName(finalMeta.Partner);
if (!string.IsNullOrWhiteSpace(partnerFolder))
{
    await EnsureFolderExistsAsync(client, driveId, zoneFolder,
   partnerFolder, ct);
    targetFolder = string.IsNullOrWhiteSpace(zoneFolder) ? partnerFolder :
    $"{zoneFolder}/{partnerFolder}";
}

var uploadUrl = string.IsNullOrEmpty(targetFolder)
    ? $"drives/{encodedDriveId}/root:/{encodedFileName}:/content"
    : $"drives/{encodedDriveId}/root:/{EncodePathForGraph(targetFolder)}/{
   encodedFileName}:/content";

using (var stream = File.OpenRead(binPath))
{
    var resp = await client.PutAsync(uploadUrl, new StreamContent(stream),
    ct);
    resp.EnsureSuccessStatusCode();
    var item = await resp.Content.ReadFromJsonAsync<JsonElement>(ct);
    sharePointItemId = item.GetProperty("id").GetString()!;
}
```

Die Partner-Unterordner werden vor dem Datei-Upload gezielt erstellt. Ungültige Zeichen werden über `SanitizeFolderName(...)` ersetzt, damit SharePoint-konforme Pfade entstehen.

4.2.1.7. Persistenz in Documents-API und Suchindex

Nach erfolgreichem SharePoint-Write registriert der Zone-Service das Dokument in der Documents-API und übergibt dabei auch `sharePointItemId` und `ownerId`. Danach wird ein interner Indexierungsaufruf an die Search-API ausgelöst.

Quellcode 4.5: Commit-Endpunkt mit Metadatenpersistenz und Indexierung

```
var (meta, storedPath, sha256, contentText, sharePointItemId) =
    await store.CommitTempToSharePointAsync(user.TenantId, user.ObjectId,
    user.DisplayName ?? "",
                                            uploadId, confirmed, ct);

var dto = new CreateDocumentDto(
    zoneId, meta.OriginalFilename, storedPath, meta.MimeType, meta.
    SizeBytes, sha256,
    meta.DocumentType, meta.DocumentYear, meta.ContractType, meta.Partner,
    meta.ExpireDate, sharePointItemId, user.ObjectId);

var regResp = await docApi.PostAsJsonAsync("/documents", dto, ct);
var created = await regResp.Content.ReadFromJsonAsync<
    CreatedDocumentResponse>(ct);

searchApi.DefaultRequestHeaders.TryAddWithoutValidation("X-Internal-Key",
    config["Services:SearchInternalApiKey"]!);
await searchApi.PostAsJsonAsync("/internal/index", new IndexDocumentDto(
    created!.id, zoneId, meta.OriginalFilename, storedPath, meta.
    DocumentType,
    meta.DocumentYear, meta.ContractType, meta.Partner, meta.
    ClassifiedAtUtc,
    contentText, new[] { user.ObjectId }), ct);
```

Die Suche wird damit direkt nach dem Upload aktualisiert; es ist kein separater Batchlauf notwendig, um neue Dokumente auffindbar zu machen.

4.2.1.8. Fehlerbehandlung und Sicherheitsaspekte im Upload-Flow

Der Flow implementiert mehrere Sicherheits- und Robustheitsmechanismen:

- Autorisierung auf allen geschützten Zone-Endpunkten über Benutzerkontext (`TenantId`, `ObjectId`),
- temporäre Entkopplung via `uploadId`, um unbestätigte Uploads nicht sofort final zu persistieren,
- typisierte Persistenzobjekte für API-Grenzen (DTOs für Dokumenterfassung und Suchindexierung),
- interne Indexierung nur mit `X-Internal-Key`,
- ACL-Übernahme in den Index über `AllowedUsers`.

Zusätzlich führen Fehler in Preflight, Commit oder Metadatenpersistenz zu klaren Benutzer-Rückmeldungen im Frontend (Toasts) und unterbrechen den Vorgang kontrolliert, statt inkonsistente Zwischenzustände zu erzeugen.

4.2.1.9. Zusammenfassung der Upload-Implementierung

Die Upload-Implementierung verbindet Benutzerführung, KI-Voranalyse und Persistenz in einem durchgängigen Prozess. Durch die Trennung in Preflight und Commit bleiben KI-Ergebnisse vor dem endgültigen Speichern überprüfbar. Datenhaltung und Suchindexierung werden danach konsistent aktualisiert.

4.2.2. Dokumenten-Klassifikation

Dieses Kapitel dokumentiert die implementierte Dokumenten-Klassifikation und beschreibt den technischen Ablauf vom API-Endpunkt bis zur Nachvalidierung der Ergebnisse.

4.2.2.1. Zweck und Ergebnisobjekt

Die Klassifikation extrahiert aus einem PDF die für die weitere Verarbeitung relevanten Metadaten und liefert diese in einem einheitlichen DTO zurück. Das Ergebnis besteht aus Dokumenttyp, Jahr, Vertragstyp, Partner und optionalem Ablaufdatum.

Quellcode 4.6: Rückgabeobjekt der Klassifikation

```
public record DocumentClassificationResultDto(
    DocumentType DocumentType,
    int? DocumentYear,
    string? ContractType,
    string? Partner,
    DateTime? ExpireDate
);
```

Damit ist das Ergebnisformat für alle aufrufenden Komponenten stabil und typisiert.

4.2.2.2. Klassifikations-Endpunkt

Die AI-API stellt den Endpunkt `POST /ai/classify` bereit. Der Controller validiert die Eingabe (nicht leer, MIME-Type PDF) und delegiert die eigentliche Klassifikationslogik an den Service `OpenAiClassifier`.

Quellcode 4.7: API-Endpunkt der Dokumenten-Klassifikation

```
[HttpPost("classify")]
public async Task<ActionResult<DocumentClassificationResultDto>> Classify(
    [FromForm] IFormFile file, CancellationToken ct)
```

```
{
    if (file.Length == 0) return BadRequest("Empty file.");
    if (!string.Equals(file.ContentType, "application/pdf",
        StringComparison.OrdinalIgnoreCase))
        return BadRequest("Only PDF supported for now.");

    await using var s = file.OpenReadStream();
    var result = await _classifier.ClassifyPdfAsync(s, file.FileName, ct);
    return Ok(result);
}
```

Die Verantwortung des Controllers bleibt damit bewusst schlank: Validierung, Delegation und Response.

4.2.2.3. Kernlogik im OpenAiClassifier

Der Service `OpenAiClassifier` enthält die eigentliche KI-Interaktion. Die Pipeline besteht aus vier Schritten:

1. Upload der PDF-Datei an die OpenAI Files API,
2. Aufruf der Responses API mit Modell, Prompt und strikt vorgegebenem JSON-Schema,
3. Parsen der strukturierten Antwort,
4. technische und fachliche Nachvalidierung der Felder.

Der wichtigste Baustein ist das Schema-gebundene Prompting, weil dadurch die Antwort in ein verlässliches, maschinenlesbares Format gezwungen wird.

Quellcode 4.8: Schema-gebundener Responses-Aufruf

```
var payload = new
{
    model = _model,
    input = new[]
    {
        new {
            role = "user",
            content = new object[]
            {
                new { type = "input_file", file_id = fileId },
                new { type = "input_text", text = prompt }
            }
        }
    },
    text = new
    {
        format = new
        {
            type = "json_schema",
            name = "doc_extraction",
            strict = true,
```

```
22                schema
23            }
24        }
25 };
26
27 using var resp = await _http.PostAsync(
28     "https://api.openai.com/v1/responses",
29     new StringContent(JsonSerializer.Serialize(payload), Encoding.UTF8, "
       application/json"),
30     ct);
```

Durch `strict = true` und das definierte Schema wird die Ausgabe stark normalisiert. Das reduziert nachgelagerte Parsingfehler.

4.2.2.4. Parsing und fachliche Nachvalidierung

Nach dem API-Aufruf wird der JSON-Text extrahiert, in die internen Typen gemappt und anschließend fachlich bereinigt. Dabei gelten unter anderem folgende Regeln:

- Jahr nur im Bereich 1900 bis 2100,
- `ContractType` und `ExpireDate` nur für `DocumentType.Vertrag`,
- leere Strings werden auf `null` normalisiert.

Quellcode 4.9: Nachvalidierung der Klassifikationsergebnisse

```
1 var typeStr = root.GetProperty("document_type").GetString()!;
2 var docType = Enum.Parse<DocumentType>(typeStr, ignoreCase: true);
3
4 int? year = null;
5 var y = root.GetProperty("year");
6 if (y.ValueKind == JsonValueKind.Number && y.TryGetInt32(out var yi))
7     year = yi;
8
9 if (year is < 1900 or > 2100) year = null;
10 if (docType != DocumentType.Vertrag)
11 {
12     contractType = null;
13     expireDate = null;
14 }
```

Diese Nachvalidierung ist zentral, damit inkonsistente Modellantworten nicht ungeprüft in den weiteren Prozess übernommen werden.

4.2.2.5. Fehlerbehandlung und Robustheit

Die Service-Methoden prüfen HTTP-Antworten zentral über `EnsureOk(...)`. Bei nicht erfolgreichem Status wird die API-Antwort inklusive Statuscode in eine Exception übernommen. Dadurch sind Fehler in Files-Upload oder Responses-Aufruf klar lokalisierbar und beobachtbar.

Zusätzlich unterstützt `ExtractFirstOutputText(...)` mehrere Antwortvarianten, insbesondere ein direktes `output_text`-Feld sowie textuelle Inhalte in der `output`-Struktur. Das macht die Verarbeitung robuster gegenüber Response-Formvarianten.

4.2.2.6. Zusammenfassung der Klassifikationsimplementierung

Die Dokumenten-Klassifikation ist als klar gekapselter Service umgesetzt:

- schlanker API-Endpunkt für PDF-Eingaben,
- schema-basierter KI-Aufruf für strukturierte Extraktion,
- typisierte Ergebnisrückgabe über `DocumentClassificationResultDto`,
- sowie explizite Nachvalidierung der KI-Ergebnisse.

Damit ist die Klassifikationslogik technisch isoliert dokumentiert und als wiederverwendeter Baustein beschrieben.

4.2.3. Dokumenten-Suche

Dieses Kapitel dokumentiert die implementierte Suchfunktion über den gesamten Request-Flow hinweg: von der Benutzerinteraktion in der Weboberfläche bis zur Ausführung der Abfrage in Azure AI Search und der Rückgabe von Treffern inklusive Facetten.

4.2.3.1. Technischer Such-Flow

Die Suche ist als mehrstufiger Ablauf umgesetzt:

1. Die Benutzerin bzw. der Benutzer gibt Suchtext und optionale Filter (Dokumenttyp, Vertragstyp, Jahr, Partner) in der Seite `Search.razor` ein.
2. Das Web-Frontend baut daraus ein `SearchRequestDto` und sendet einen `POST` an den Search-Service (`/search`).
3. Der Search-Service liest die Benutzeridentität (`oid`) aus dem Access Token und erzwingt damit eine benutzerbezogene Zugriffskontrolle.
4. Der `AzureSearchQueryService` erstellt `SearchOptions` (Paging, Facets, Filter) und führt die Abfrage gegen den Azure-Search-Index aus.
5. Die Treffer und Facetten werden in ein `SearchResponseDto` gemappt und an das Frontend zurückgegeben.
6. Das Frontend aktualisiert Trefferliste und Filteroptionen dynamisch anhand der zurückgegebenen Facetten.

4.2.3.2. Frontend: Aufbau und Versand des Such-Requests

Die zentrale Suchmethode im Frontend befindet sich in `Search.razor`. Vor dem eigentlichen Suchaufruf wird eine Synchronisation mit der Zone-API ausgelöst. Anschließend wird das DTO aufgebaut und an den Suchendpunkt gesendet.

Quellcode 4.10: Suchen im Frontend mit DTO-Aufbau und API-Aufruf

```
private async Task RunSearch()
{
    await ZoneApi.PostAsync("/zones/sync-all", null);

    var req = new SearchRequestDto(
        Query: string.IsNullOrWhiteSpace(_query) ? null : _query.Trim(),
        DocumentYear: int.TryParse(_documentYear, out var y) ? y : null,
        DocumentType: Enum.TryParse<DocumentType>(_documentType, true, out var dt) ? dt : null,
        ContractType: string.IsNullOrWhiteSpace(_contractType) ? null : _contractType,
        Partner: string.IsNullOrWhiteSpace(_partner) ? null : _partner,
        Skip: 0,
        Size: 50
    );

    var resp = await SearchApi.PostAsJsonAsync("/search", req);
    resp.EnsureSuccessStatusCode();
}
```

Die Implementierung ist bewusst defensiv: Leere Eingaben werden als `null` übertragen, numerische und Enum-Filter werden nur bei erfolgreicher Konvertierung gesetzt. Dadurch bleiben Request und Filterlogik stabil, auch wenn einzelne UI-Felder leer bleiben.

4.2.3.3. Suchmodi und Filterkombinationen

Die Suche unterstützt mehrere praktische Nutzungsarten:

- Volltextsuche über `Query` (z. B. Begriff aus Dateiname oder Dokumentinhalt),
- reine Filterabfrage ohne Suchtext (z. B. nur Jahr, Partner oder Dokumenttyp),
- kombinierte Suche aus Volltext und Filtern (z. B. Suchwort plus Jahr und Partner),
- gezielte Eingrenzung über Vertragsart (`ContractType`) bei Vertragsdokumenten.

Technisch wird bei leerem Suchtext automatisch `"*"` als Suchausdruck verwendet. Dadurch kann das System auch dann Ergebnisse liefern, wenn ausschließlich nach Metadaten wie Jahr, Partner oder Dokumenttyp gefiltert wird.

4.2.3.4. Search API: Authentifizierter Endpunkt

Der Search-Service stellt die fachliche Suchoperation über `POST /search` bereit. Der Endpunkt ist geschützt (`RequireAuthorization()`) und extrahiert die Benutzerkennung aus dem JWT-

Token. Ohne gültige `oid` wird der Request abgelehnt.

Quellcode 4.11: Search-Endpunkt mit Benutzerkontext

```
app.MapPost("/search", async (SearchRequestDto req, HttpContext ctx,
                              AzureSearchQueryService svc,
    CancellationToken ct) =>
{
    var oid = ctx.User.FindFirst("oid")?.Value
              ?? ctx.User.FindFirst(
                  "http://schemas.microsoft.com/identity/claims/
    objectidentifier"
                  )?.Value;

    if (string.IsNullOrWhiteSpace(oid)) return Results.Unauthorized();

    var result = await svc.SearchAsync(req, oid, ct);
    return Results.Ok(result);
}).RequireAuthorization();
```

Damit ist sichergestellt, dass jeder Suchaufruf mit einem konkreten Benutzerkontext verknüpft ist und die nachgelagerte Filterlogik diesen Kontext verwenden kann.

4.2.3.5. Query-Service: Paging, Facets und Filter

Die eigentliche Suchlogik liegt im `AzureSearchQueryService`. Dort werden Suchtext, Paging und Facetten konfiguriert. Für leere Suchanfragen wird ein Wildcard-Searchtext (`"*"`) verwendet, damit auch reine Filterabfragen möglich sind.

Quellcode 4.12: SearchOptions und Filteraufbau im Query-Service

```
var searchText = string.IsNullOrWhiteSpace(req.Query) ? "*" : req.Query;

var opt = new SearchOptions
{
    IncludeTotalCount = true,
    Skip = Math.Max(0, req.Skip),
    Size = Math.Clamp(req.Size, 1, 100)
};

opt.Facets.Add("documentYear,count:50");
opt.Facets.Add("documentType,count:50");
opt.Facets.Add("partner,count:50");

opt.Filter = BuildFilter(req, callerOid,
    includeDocumentType: true,
    includeDocumentYear: true,
    includeContractType: true,
    includePartner: true);

var resp = await _client.SearchAsync<SearchDocument>(searchText, opt, ct);
```

Die Limits (`Size` zwischen 1 und 100) schützen den Service vor übergroßen Antworten. Gleichzeitig liefern Facetten die Datenbasis für dynamische Filteroptionen im UI.

Die wichtigste Sicherheitsregel ist im Filteraufbau implementiert: Ergebnisse werden immer auf Dokumente eingeschränkt, die für den aktuellen Benutzer freigegeben sind.

Quellcode 4.13: Benutzerbezogener OData-Filter

```
private static string BuildFilter(SearchRequestDto req, string callerOid,
    ...)
{
    var parts = new List<string>();

    parts.Add($"allowedUsers/any(u: u eq '{EscapeOData(callerOid)}')");

    if (req.DocumentYear is not null)
        parts.Add($"documentYear eq {req.DocumentYear.Value}");
    if (req.DocumentType is not null)
        parts.Add($"documentType eq '{EscapeOData(req.DocumentType.Value.
    ToString())}'");
    if (!string.IsNullOrWhiteSpace(req.Partner))
        parts.Add($"partner eq '{EscapeOData(req.Partner)}'");

    return string.Join(" and ", parts);
}
```

Durch die Kombination aus ACL-Bedingung und fachlichen Filtern entsteht ein Security-First-Ansatz: Ein Dokument wird nur dann zurückgegeben, wenn sowohl die Berechtigung als auch die Suchkriterien erfüllt sind.

4.2.3.6. Rückgabe und UI-Aktualisierung

Nach der Suchausführung werden die Index-Dokumente auf `SearchHitDto`-Objekte gemappt. Zusätzlich werden Facettenwerte in Bucket-Strukturen überführt. Das Frontend nutzt diese Buckets, um Filterlisten (z. B. Jahre, Vertragstypen, Partner) mit Trefferanzahl zu aktualisieren.

Dadurch entsteht eine adaptive Suchoberfläche: Filter zeigen nur Werte an, die im aktuellen Ergebnisraum tatsächlich vorkommen. Das reduziert ungültige Kombinationen und verbessert die Bedienbarkeit bei großen Dokumentmengen.

4.2.3.7. Zusammenfassung der Implementierung

Die implementierte Dokumentensuche kombiniert:

- einen authentifizierten API-Endpunkt,
- benutzerbezogene Zugriffskontrolle direkt in der Query,
- kombinierbare Volltext- und Metadatenfilter,

- serverseitige Facettenaggregation,
- sowie eine dynamische UI-Nachführung der Filteroptionen.

Damit ist die Suche sowohl funktional als auch sicherheitstechnisch in die Gesamtarchitektur integriert und für den produktiven Einsatz mit mandanten-/benutzerbezogenen Daten geeignet.

4.2.4. Benutzer- und Rollenverwaltung

4.2.5. System-Logging und Monitoring

4.2.6. API-Endpunkte

4.2.7. Fehlerbehandlung und Ausnahmen

4.2.8. Sicherheitsfunktionen

5. Beurteilung

5.1. Bewertung der Implementierung

5.2. Erfüllung der Forschungsfragen

5.3. Kritische Reflexion und Grenzen

6. Zusammenfassung und Ausblick

6.1. Zusammenfassung

Zusammenfassend war diese Diplomarbeit ein sehr lehrreiches Projekt, bei dem wir viele neue Erfahrungen gemacht haben. ...

6.2. Ausblick

I. Literaturverzeichnis

[1] SER Group: *Document Classification – mit KI zum optimalen Input Management*, Januar 2024. Online in Internet: URL: `https://www.sergroup.com/de/knowledge-center/blog/document-classification-mit-ki.html`.

[2] Shaip: *KI-basierte Dokumentenklassifizierung – Vorteile, Prozesse und Anwendungsfälle*, Juli 2025. Online in Internet: URL: `https://de.shaip.com/blog/ai-based-document-classification/`.

[3] Griesch, Leon, Leon Görgen und Kurt Sandkuhl: *Ki-als-service: Vergleich von plattformen zur dokumentenklassifikation.* In: *INFORMATIK 2024*, Seiten 1505–1518. Gesellschaft für Informatik eV, 2024.

[4] Microsoft Learn: *Use azure api management to front azure openai and other language models.* Online in Internet: URL: `https://learn.microsoft.com/en-us/azure/api-management/ai-gateway-capabilities` (Zugriff: 08.03.2026).

[5] Microsoft Learn: *Load balance azure openai endpoints with azure api management.* Online in Internet: URL: `https://learn.microsoft.com/en-us/azure/architecture/ai-ml/guide/azure-openai-gateway-multi-backend` (Zugriff: 08.03.2026).

[6] Microsoft Learn: *Asynchronous request-reply pattern.* Online in Internet: URL: `https://learn.microsoft.com/en-us/azure/architecture/patterns/async-request-reply` (Zugriff: 08.03.2026).

[7] Microsoft Learn: *Retry pattern.* Online in Internet: URL: `https://learn.microsoft.com/en-us/azure/architecture/patterns/retry` (Zugriff: 08.03.2026).

[8] Microsoft Learn: *Circuit breaker pattern.* Online in Internet: URL: `https://learn.microsoft.com/en-us/azure/architecture/patterns/circuit-breaker` (Zugriff: 08.03.2026).

[9] OpenAI: *Latency optimization.* Online in Internet: URL: `https://platform.openai.com/docs/guides/latency-optimization` (Zugriff: 08.03.2026).

[10] OpenAI: *Prompt caching 101.* Online in Internet: URL: `https://cookbook.openai.com/examples/prompt_caching101` (Zugriff: 08.03.2026).

[11] Evoluce GmbH: *Merkmalsextraktion mit KI: Relevantes erkennen, Unwichtiges ausblenden.* Online in Internet: URL: `https://evoluce.de/merkmalsextraktion/` (Zugriff: 03.02.2026). Veröffentlichungsdatum auf der Seite nicht angegeben.

[12] Barnard, Joel: *Was ist Embedding?* Online in Internet: URL: `https://www.ibm.com/de-de/think/topics/embedding` (Zugriff: 03.02.2026).

[13] Dascalescu, Dan and Zain Hasan: *Vector embeddings explained*, January 2023. Online in Internet: URL: `https://weaviate.io/blog/vector-embeddings-explained` (Zugriff: 03.02.2026).

[14] Elastic: *Text classification.* Online in Internet: URL: `https://www.elastic.co/de/what-is/text-classification` (Zugriff: 03.02.2026).

[15] Google Cloud: *What are convolutional neural networks (cnns)?* Online in Internet: URL: `https://cloud.google.com/discover/what-are-convolutional-neural-networks` (Zugriff: 03.02.2026).

[16] Harley, Adam: *Rvl-cdip: A large-scale dataset for document classification.* Online in Internet: URL: `https://adamharley.com/rvl-cdip/` (Zugriff: 03.02.2026).

[17] V7 Labs: *Vision transformers (vit): A practical guide.* Online in Internet: URL: `https://www.v7labs.com/blog/vision-transformer-guide` (Zugriff: 03.02.2026).

[18] Hugging Face: *Layoutlm.* Online in Internet: URL: `https://huggingface.co/docs/transformers/en/model_doc/layoutlm` (Zugriff: 03.02.2026).

[19] Azure Samples: *Document classification with gpt vision (notebook).* Online in Internet: URL: `https://github.com/azure-samples/azure-ai-document-processing-samples/blob/main/samples/python/classification/document-classification-gpt-vision.ipynb` (Zugriff: 03.02.2026).

[20] Tesseract OCR Contributors: *Improvequality.* Online in Internet: URL: `https://tesseract-ocr.github.io/tessdoc/ImproveQuality.html` (Zugriff: 05.03.2026).

[21] Xu, Yiheng, Minghao Li, Lei Cui, Shaohan Huang, Furu Wei, and Ming Zhou: *Layoutlm: Pre-training of text and layout for document image understanding.* In *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, pages 1192–1200, 2020. Online in Internet: URL: `https://arxiv.org/abs/1912.13318` (Zugriff: 05.03.2026).

[22] Xu, Yiheng, Tengchao Lv, Lei Cui, Furu Wei, Guoyin Wang, and Yiming Lu: *Layoutlmv2: Multi-modal pre-training for visually-rich document understanding.* In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing*, pages 2579–2591, 2021. Online in Internet: URL: `https://arxiv.org/abs/2012.14740` (Zugriff: 05.03.2026).

[23] scikit-learn developers: *train_test_split.* Online in Internet: URL: `https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.train_test_split.html` (Zugriff: 05.03.2026).

[24] scikit-learn developers: *Cross-validation: evaluating estimator performance.* Online in Internet: URL: `https://scikit-learn.org/stable/modules/cross_validation.html` (Zugriff: 05.03.2026).

[25] Manning, Christopher D., Prabhakar Raghavan, and Hinrich Schütze: *Introduction to Information Retrieval.* Cambridge University Press, 2008. Online in Internet: URL: `https://nlp.stanford.edu/IR-book/` (Zugriff: 05.03.2026).

[26] Elastic: *Query and filter context.* Online in Internet: URL: `https://www.elastic.co/guide/en/elasticsearch/reference/current/query-filter-context.html` (Zugriff: 05.03.2026).

[27] Stanford NLP Group: *Tf-idf weighting.* Online in Internet: URL: `https://nlp.stanford.edu/IR-book/html/htmledition/tf-idf-weighting-1.html` (Zugriff: 05.03.2026).

[28] Robertson, Stephen and Hugo Zaragoza: *The probabilistic relevance framework: Bm25 and beyond.* Foundations and Trends in Information Retrieval, 3(4):333–389, 2009. Online in Internet: URL: `https://www.nowpublishers.com/article/Details/INR-019` (Zugriff: 05.03.2026).

[29] Apache Lucene: *Package org.apache.lucene.search.similarities*. Online in Internet: URL: https://lucene.apache.org/core/9_9_1/core/org/apache/lucene/search/similarities/package-summary.html (Zugriff: 05.03.2026).

[30] Elastic: *knn search*. Online in Internet: URL: https://www.elastic.co/guide/en/elasticsearch/reference/current/knn-search.html (Zugriff: 05.03.2026).

[31] Malkov, Yu A. and Dmitry A. Yashunin: *Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs*. IEEE Transactions on Pattern Analysis and Machine Intelligence, 42(4):824–836, 2020. Online in Internet: URL: https://arxiv.org/abs/1603.09320 (Zugriff: 05.03.2026).

[32] Elastic: *Reciprocal rank fusion*. Online in Internet: URL: https://www.elastic.co/guide/en/elasticsearch/reference/current/rrf.html (Zugriff: 05.03.2026).

[33] Elastic: *Boolean query*. Online in Internet: URL: https://www.elastic.co/guide/en/elasticsearch/reference/current/query-dsl-bool-query.html (Zugriff: 05.03.2026).

[34] Elastic: *Bucket aggregations*. Online in Internet: URL: https://www.elastic.co/guide/en/elasticsearch/reference/current/search-aggregations-bucket.html (Zugriff: 05.03.2026).

[35] Elastic: *Terms aggregation*. Online in Internet: URL: https://www.elastic.co/guide/en/elasticsearch/reference/current/search-aggregations-bucket-terms-aggregation.html (Zugriff: 05.03.2026).

[36] Elastic: *Date histogram aggregation*. Online in Internet: URL: https://www.elastic.co/guide/en/elasticsearch/reference/current/search-aggregations-bucket-datehistogram-aggregation.html (Zugriff: 05.03.2026).

[37] Apache Solr: *Faceting*. Online in Internet: URL: https://solr.apache.org/guide/solr/latest/query-guide/faceting.html (Zugriff: 05.03.2026).

[38] Tsolkas, Alexander und Klaus Schmidt: *Rollen und Berechtigungskonzepte: Identity-und Access-Management im Unternehmen*. Springer-Verlag, 2017.

[39] StudySmarter: *Skalierbare Architekturen: Definition, Arten & Beispiele*, 2024. Online im Internet: URL: https://www.studysmarter.de/schule/informatik/technische-informatik/skalierbare-architekturen/, abgerufen am 27. Januar 2024.

[40] Wikipedia-Autoren: *Plattformunabhängigkeit — Wikipedia, Die freie Enzyklopädie*, Dezember 2024. Online im Internet: URL: https://de.wikipedia.org/wiki/Plattformunabh%C3%A4ngigkeit, abgerufen am 27. Januar 2026.

[41] Microsoft Support: *SharePoint – Hilfe und Lernen*, 2024. https://support.microsoft.com/de-de/sharepoint, Zugegriffen am: 02.02.2026.

[42] Microsoft Learn: *Übersicht über Websites und Websitesammlungen in SharePoint Server*, 2023. https://learn.microsoft.com/de-de/sharepoint/sites/sites-and-site-collections-overview, Zugegriffen am: 02.02.2026.

[43] Microsoft Corporation: *SharePoint – Hilfe und Lernen*, 2026. https://support.microsoft.com/de-de/sharepoint, Zugriff am 8. März 2026.

[44] Microsoft: *Was ist Microsoft Entra?*, 2025. https://learn.microsoft.com/de-de/entra/fundamentals/what-is-entra, Abgerufen am 2. Februar 2026.

[45] abc: *DB-Engine Ranking*, März 2016. Online in Internet: URL: http://db-engines.com/de/ranking.

II. Abbildungsverzeichnis

3.1. Übersicht der Vektordarstellung von Embeddings [13] 18
3.2. Übersicht der Microsoft Entra Produktfamilie [44] 33
3.3. Integration von On-Premises- und Cloud-Anwendungen über Microsoft Entra ID [44] 35

III. Tabellenverzeichnis

3.1. Textklassifizierung vs. Dokumentenklassifizierung [2] 11
3.2. Wichtige Begriffe der Merkmalsextraktion [11] 16
3.3. Relevante Algorithmen zur Merkmalsextraktion [11] 16
3.4. Relevante Objekte für Embeddings [12] 19
3.5. Qualitätsziele in der Dokumentaufbereitung [20] [21] 22
3.6. Vergleich von Suchansätzen in Dokumentsystemen [28] [30] [32] 27
3.7. Komponenten einer Berechtigung 27
3.8. Merkmale von Softwaresystemen 29
3.9. Techniken zur Leistungsoptimierung 29
3.10. Überblick über SharePoint-Produkte 31

A.1. Kapitelverzeichnis 59
A.2. Arbeitstagebuch Toifl 59
A.3. Arbeitstagebuch Schaider 59

IV. Quellcodeverzeichnis

4.1. Zone-Erstellung mit SharePoint-Ordnerinitialisierung 37
4.2. Preflight- und Commit-Steuerung im UploadQueueService 38
4.3. Preflight-Endpunkt im Zone-Service 38
4.4. Commit-Logik mit SharePoint-Upload 40
4.5. Commit-Endpunkt mit Metadatenpersistenz und Indexierung 41
4.6. Rückgabeobjekt der Klassifikation 42
4.7. API-Endpunkt der Dokumenten-Klassifikation 42
4.8. Schema-gebundener Responses-Aufruf 43
4.9. Nachvalidierung der Klassifikationsergebnisse 44
4.10. Suchen im Frontend mit DTO-Aufbau und API-Aufruf 46
4.11. Search-Endpunkt mit Benutzerkontext 47
4.12. SearchOptions und Filteraufbau im Query-Service 47
4.13. Benutzerbezogener OData-Filter 48

A. Anhang

A.1. Arbeitsteilung

Kurze Beschreibung, wer was gemacht hat (Überblick).

A.2. Kapitelverzeichnis

Kapitel	Editor
?? ??	Max Mustermann
?? ??	Mex Musterjuan

Tabelle A.1.: Kapitelverzeichnis

A.3. Projekttagebücher

A.3.1. Projekttagebuch Max Mustermann

Tag	Zeit	kumulativ	Fortschritt
Mo 28.11.16	2h	2h	Besprechnung der Programmanforderungen
Di 29.11.16	3h	5h	Datenbankmodell erstellt
Mi 30.11.16	1h	6h	Datenbankmodellüberarbeitet
Do 01.12.16	3h	9h	Pflichtenheft erstellt

Tabelle A.2.: Arbeitstagebuch Toifl

A.3.2. Projekttagebuch Mex Musterjuan

Tag	Zeit	kumulativ	Fortschritt
Mo 28.11.16	2h	2h	Besprechnung der Programmanforderungen

Tabelle A.3.: Arbeitstagebuch Schaider

A.4. Besprechungsprotokolle

...Hier können auch pdf Dateien eingebunden werden!

Betreuungsprotokoll zur Diplomarbeit lfd. Nr.:

Themenstellung:
Kandidaten/Kandidatinnen:

Jahrgang:
Betreuer/in:
Ort:
Datum:
Zeit:

Besprechungsinhalt:

Name	Notiz

Aufgaben:

Name	Notiz	zu erledigen bis

A.5. Datenträgerbeschreibung